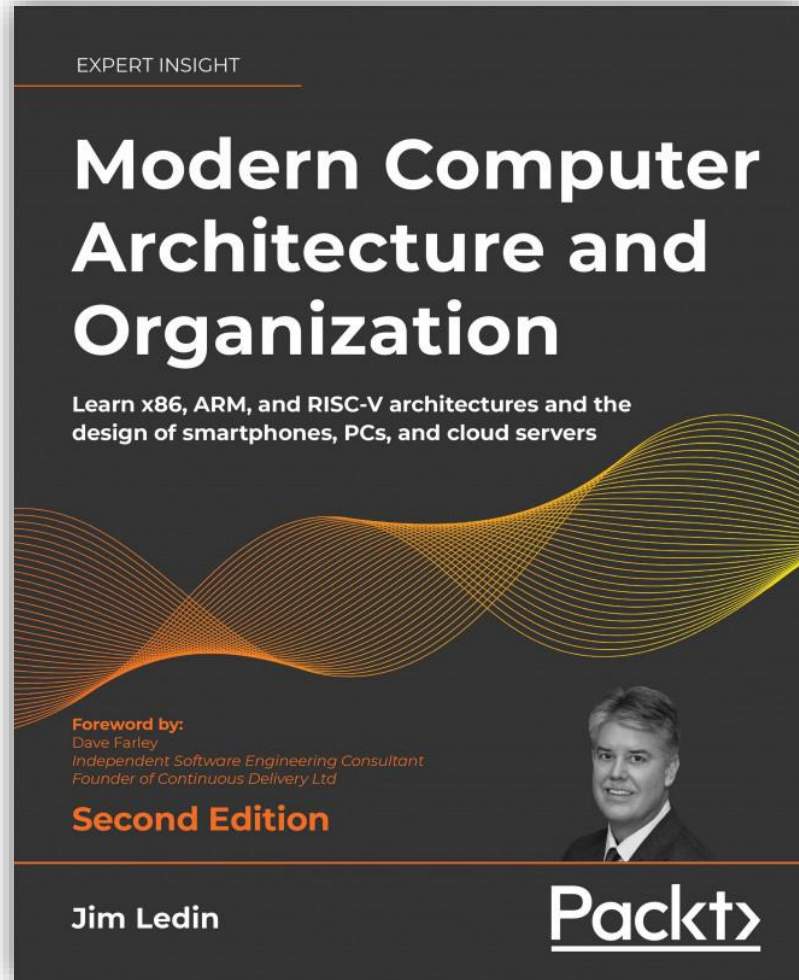


Computer Architecture

Chapter 1 – Introducing Computer Architecture



VIVES University of Applied Sciences Bachelor in electronics-ICT



— **Computer Architecture**

Chapter 1: Introducing Computer Architecture

Contents

1. **Introduction**
2. The evolution of automated computing devices
 - Charles Babbage's Analytical Engine
 - ENIAC
 - IBM PC
 - The iPhone
3. Moore's law
4. Computer architecture
 - The 6502 microprocessor
 - The 6502 instruction set

— Computer Architecture

Chapter 1: Introducing Computer Architecture

1. Introduction

- The architectures of automated computing systems has evolved from **mechanical systems** constructed nearly two centuries ago to the broad array of **modern electronic computing technologies** we use directly and indirectly every day.
- In the 1980s, during the early days of personal computing, students and technical professionals eager to learn about computing devices had a limited range of subject matter available for this purpose.
- If they had a computer of their own, it might have been an **IBM PC** or an **Apple II**. If they worked for an organization with a computing facility, they might have used an **IBM mainframe** or a **Digital Equipment Corporation VAX minicomputer**.
- Today, numerous specialized computing architectures exist to address widely varying user needs. We carry **miniature computers in our pockets** and purses that can place phone calls, record video, and function as full participants on the Internet.
- **Personal computers** remain popular in a format outwardly similar to the PCs of past decades.
- **Today's PCs**, however, are orders of magnitude more capable than the early generations in terms of computing power, memory size, disk space, graphics performance, and communication capability.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

1. Introduction

- Companies offering **web services** to hundreds of millions of users construct vast warehouses filled with **thousands of tightly coordinated computer systems** capable of responding to a constant stream of requests with extraordinary speed and precision.
- **Machine learning systems** are trained through the analysis of enormous quantities of data to perform complex activities, such as driving automobiles.
- This chapter begins by presenting **a few key historical computing devices** and the leaps in technology associated with them. We will then examine some modern-day trends related to technological advances and introduce the basic concepts of computer architecture, including a close look at the **6502 microprocessor**.
- These following topics will be covered:
 - The evolution of automated computing devices
 - Moore's law
 - Computer architecture

— **Computer Architecture**

Chapter 1: Introducing Computer Architecture

Contents

1. Introduction
- 2. The evolution of automated computing devices**
 - Charles Babbage's Analytical Engine
 - ENIAC
 - IBM PC
 - The iPhone
3. Moore's law
4. Computer architecture
 - The 6502 microprocessor
 - The 6502 instruction set

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

- Although a working model of the Analytical Engine was never constructed, the detailed notes [Charles Babbage](#) developed from 1834 until his death in 1871 described a **computing architecture** that appeared to be both workable and complete.
- The **Analytical Engine** was intended to serve as a **general-purpose programmable computing device**. The design was **entirely mechanical** and was to be constructed largely of brass. It was designed to be driven by a shaft powered by a steam engine.
- Unlike most modern computers, the Analytical Engine represented numbers in **signed decimal form**. The decision to use **base-10 numbers** rather than the base-2 logic of most modern computers was the result of a fundamental difference between mechanical technology and digital electronics.
- It is straightforward to construct **mechanical wheels** with **ten positions**, so Babbage chose the human-compatible base-10 format because it was not significantly more technically challenging than using some other number base.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

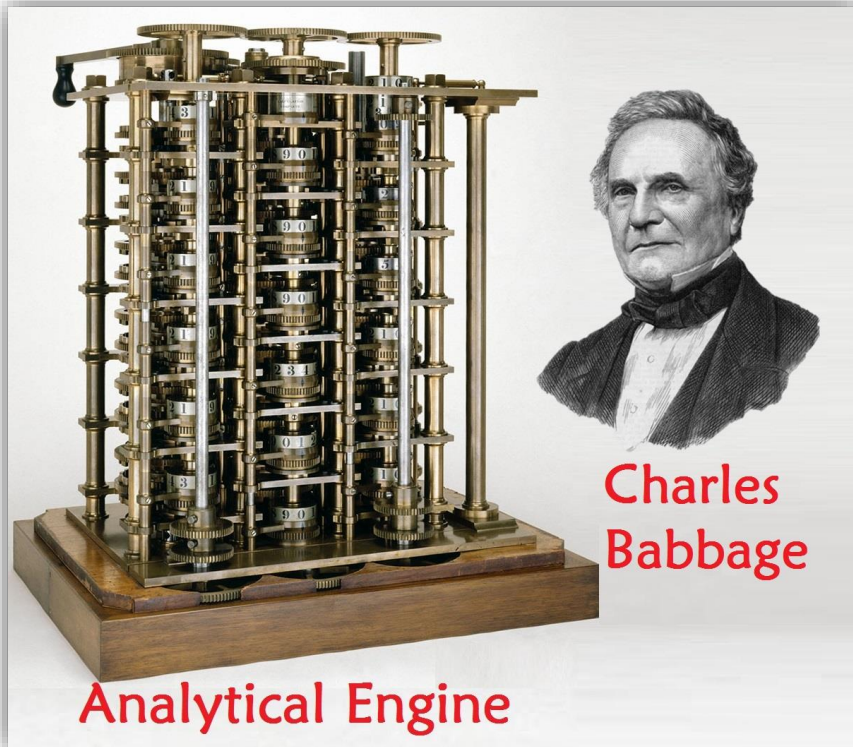


Figure 1.1: Charles Babbage's Analytical Engine

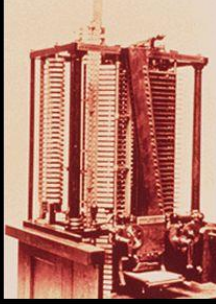
	First General-Purpose Automated Digital Computer Charles Babbage 1791-1871	
Charles Babbage Analytical Engine <u>Charles Babbage, its father.</u> • Analytical Engine • It was never completed by Babbage, due to a lack of standardized parts. • English mathematician • Frustrated genius • One of the most interesting characters in the history of computers.		

Figure 1.2: Charles Babbage's Analytical Engine

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

- All **numbers** in the Analytical Engine consisted of **40 decimal digits**. The large number of digits was likely selected to reduce problems with **numerical overflow**.
- The Analytical Engine **did *not* support floating-point mathematics**.
- Each number was stored on a **vertical axis** containing **40 wheels**, with each wheel capable of resting in ten positions corresponding to the **digits 0-9**.
- A **41st number wheel** contained the **sign**: any **even** number on this wheel represented a **positive** sign and any **odd** number represented a **negative** sign.
- The Analytical Engine axis was somewhat analogous to the register used in modern processors except the readout of an axis was destructive. If it was necessary to retain an axis's value after it had been read, **another axis** had to **store a copy of the value**.
- Numbers were transferred from one axis to another, or used in computations, by engaging a gear with each digit wheel and rotating the wheel to read out the numerical value. The **axes** serving as **system memory** were referred to collectively as the *store*.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

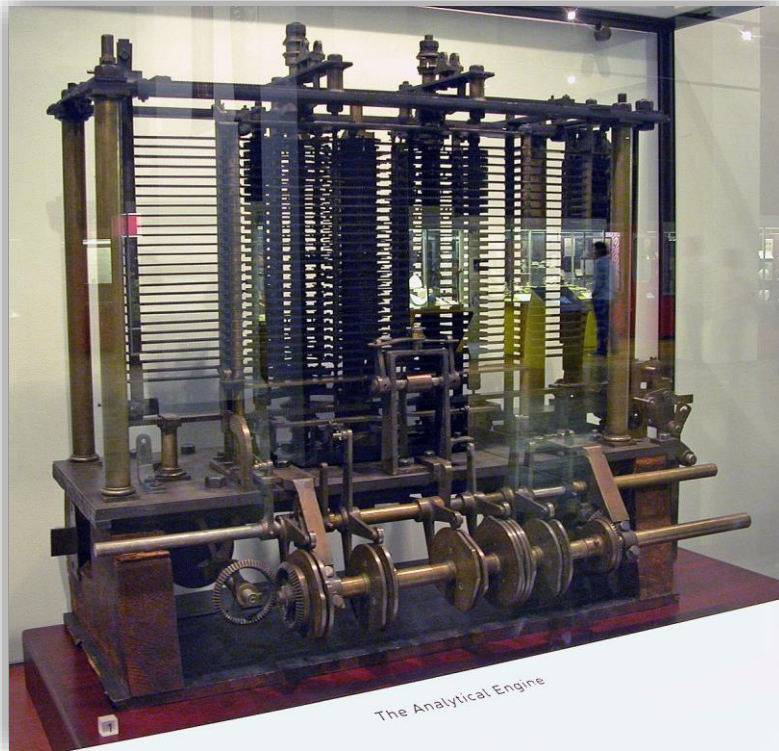


Figure 1.3: Charles Babbage's Analytical Engine



Figure 1.4: wheels representing digits 0-9

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

- The **addition** of **two numbers** used a process somewhat similar to the method of addition taught to schoolchildren.
- Assume a number stored on one axis, let's call it the **addend**, was to be added to a number on another axis, let's call it the **accumulator**. The machine would connect each addend digit wheel to the corresponding accumulator digit wheel through a train of gears.
- It would then simultaneously rotate each addend digit downward to **zero** while driving the accumulator digit an equivalent rotation in the increasing direction.
- If an accumulator digit wrapped around from nine to zero, the next most significant accumulator digit would increment by one. This **carry** operation would propagate across as many digits as needed (think of adding 1 to 999,999).
- By the end of the process, the **addend** axis would hold the **value zero** and the **accumulator** axis would hold the **sum** of the two numbers.
- The propagation of carries from one digit to the next was the most mechanically complex part of the addition process.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

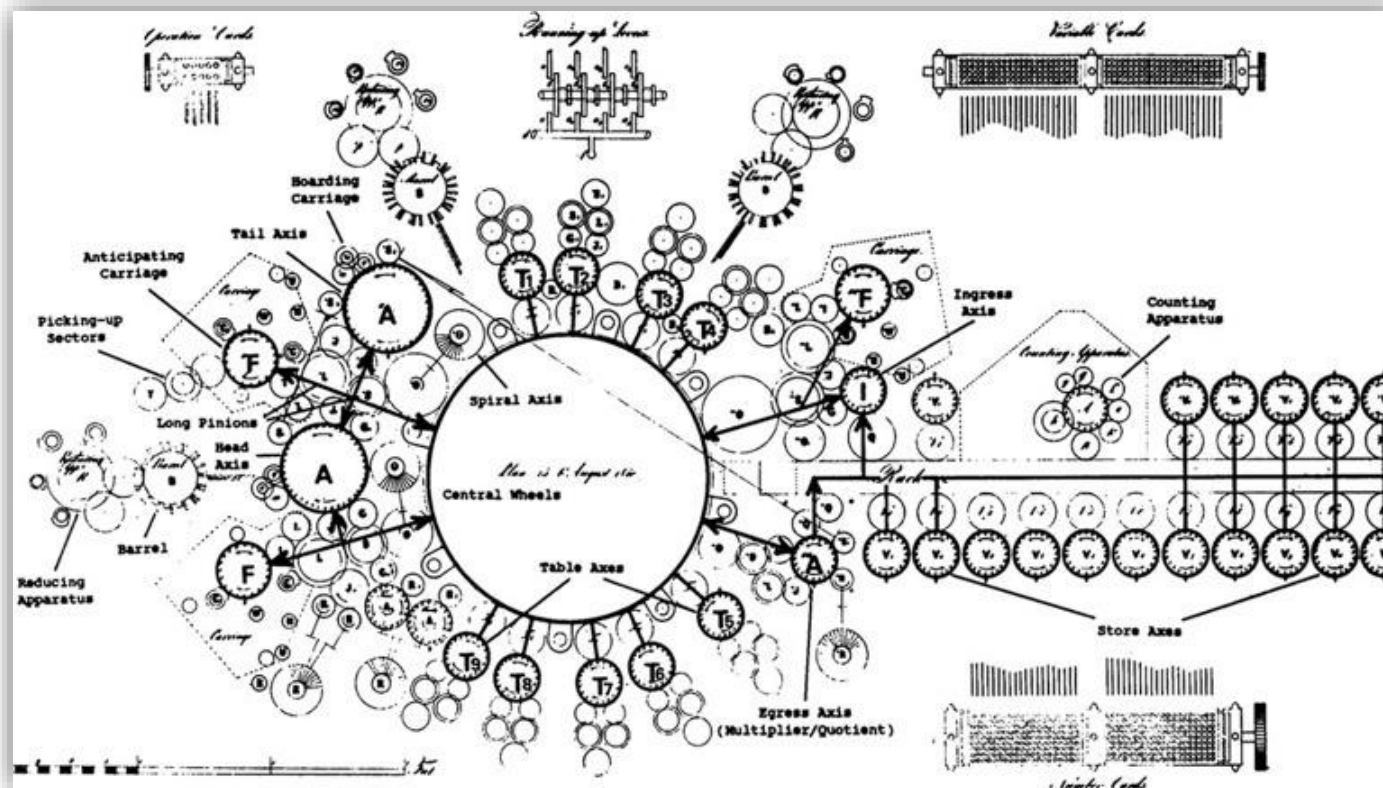


Figure 1.5: mechanical design of Charles Babbage's Analytical Engine

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

- Operations in the Analytical Engine were sequenced by music box-like rotating barrels in a construct called the **mill**, which is analogous to the *control unit* of a modern CPU.
- Each Analytical Engine **instruction** was encoded in a **vertical row of locations** on the barrel where the presence or absence of a stud at a particular location either engaged a section of the Engine's machinery or left the state of that section unchanged.
- Based on Babbage's hypothesized execution speed, the addition of two 40-digit numbers, including the propagation of carries, would take about **three seconds**.
- Babbage conceived several important concepts for the Engine that remain relevant today.
- His design supported a degree of **parallel processing** that accelerated the computation of series of values for output as numerical tables.
- Mathematical operations such as addition supported a form of **pipelining**, in which sequential operations on different data values **overlapped in time**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

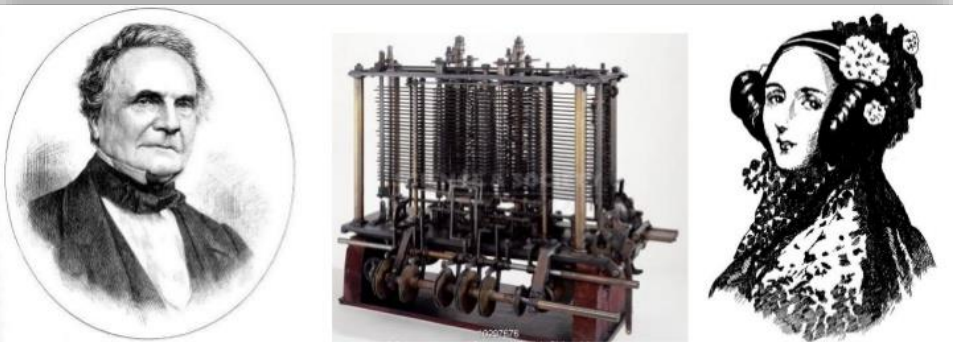
2. The evolution of automated computing devices: Charles Babbage's Analytical Engine

- Babbage was well aware of the **complexities** associated with **mechanical devices** such as **friction**, **gear backlash**, and **wear over time**.
- To prevent errors caused by these effects, the Engine incorporated mechanisms called **lockings** that were applied during data transfers across axes. The lockings forced the number wheels into **valid positions** and **prevented accumulated errors** from allowing a wheel to drift to an incorrect value.
- The use of lockings is analogous to the **amplification** of **potentially weak input signals** to produce stronger outputs by the **digital logic gates** in modern processors.
- The Analytical Engine was programmed using **punched cards** and supported **branching operations** and **nested loops**.
- The most complex program for the Analytical Engine was developed by **Ada Lovelace** to compute the **Bernoulli numbers**.
- Babbage constructed a trial model of a portion of the Analytical Engine mill, which is currently on display at the **Science Museum in London**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: Charles Babbage's Analytical Engine



Charles Babbage and Augusta Ada King, Countess of Lovelace

The designer of the analytical engine
and its programmer

Figure 1.6: Charles Babbage & Ada Lovelace

Directive Card	Operation Card	Action
1st	...	Places a on column 1 of Store
2nd	...	" b " 2 "
3rd	...	" c " 3 "
4th	...	" d " 4 "
5th	...	Brings a from Store to Mill
6th	...	" b " "
...	1	Multiplies a and $b = p$
7th	...	Takes p to column 5 of Store where it is kept for use and record
8th	...	Brings p into Mill
9th	...	Brings c into Mill
...	2	Adds p and $c = q$
10th	...	Takes q to column 6 of Store
11th	...	Brings d into Mill
12th	...	" q " "
...	3	Multiplies $d \times q = p_2$
13th	...	Takes p_2 to column 7 of Store
14th	...	Takes p_2 to printing or stereo-moulding apparatus

A table with cards and actions for the formula $(ab+c)d$

Figure 1.7: algorithm for calculating $(ab+c).d$

- YouTube video (False Dawn: The Babbage Engine): <https://www.youtube.com/watch?v=XSkGY6LchJs>

— Computer Architecture

Chapter 1: Introducing Computer Architecture

Contents

1. Introduction
2. **The evolution of automated computing devices**
 - Charles Babbage's Analytical Engine
 - **ENIAC**
 - IBM PC
 - The iPhone
3. Moore's law
4. Computer architecture
 - The 6502 microprocessor
 - The 6502 instruction set

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: ENIAC

- ENIAC, the **Electronic Numerical Integrator and Computer**, was completed in **1945** and was the **first programmable general-purpose electronic computer**.
- The system consumed **150 kilowatts** of electricity, occupied **1,800 square feet of floor space**, and **weighed 27 tons**.
- The design was based on **vacuum tubes, diodes, and relays**.
- ENIAC contained over **17,000 vacuum tubes** that functioned as **switching elements**.
- Similar to the Analytical Engine, it used **base-10 representation** of ten-digit decimal numbers implemented using **ten-position ring counters** (the ring counter will be discussed in Chapter 2, Digital Logic).
- **Input data** was received from an **IBM punch-card reader** and the **output** of computations was sent to a **card punch machine**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: ENIAC

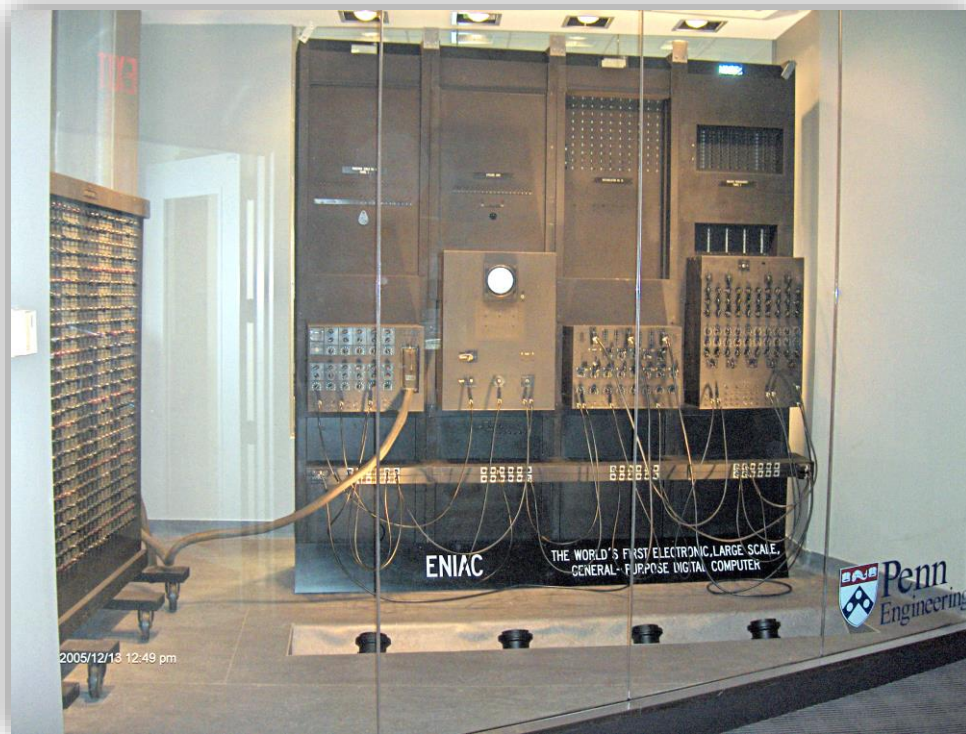


Figure 1.8: ENIAC computer

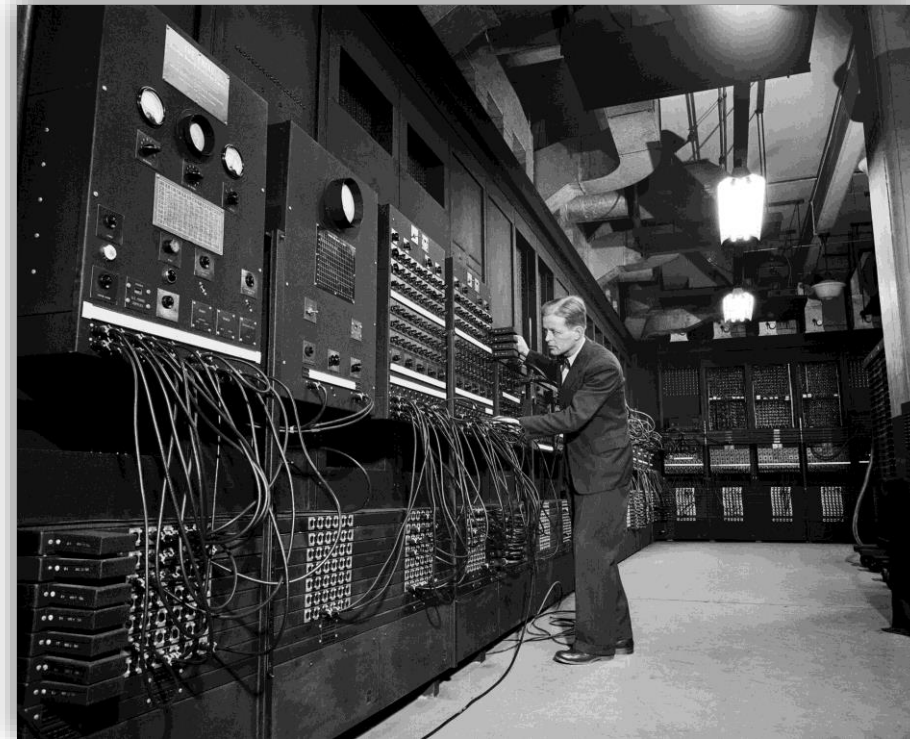


Figure 1.9: ENIAC computer

- YouTube video (ENIAC: The First Computer): https://www.youtube.com/watch?v=k4oGI_dNaPc

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: ENIAC

- The **ENIAC architecture** was capable of **complex sequences of processing steps** including **loops, branches, and subroutines**.
- The system had **20 ten-digit accumulators** that were similar to **registers** in modern computers.
- However, it did not initially have any memory storage beyond the accumulators. If intermediate values were required for use in later computations, they had to be written to **punch cards** and read back in when needed.
- ENIAC could perform about **385 multiplications per second**.
- ENIAC programs consisted of **plugboard wiring** and **switch-based function tables**.
- Programming the system was an arduous process that often took the team of talented female programmers weeks to complete.
- **Reliability** was a problem, as **vacuum tubes failed regularly**, requiring troubleshooting on a day-to-day basis to isolate and replace failed tubes.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: ENIAC

- In 1948, ENIAC was improved by adding the ability to **program** the system **via punch cards** rather than plugboards.
- This improvement greatly **enhanced the speed** with which programs could be developed.
- As a consultant for this upgrade, **John von Neumann** proposed a processing architecture based on a **single memory region** containing **program instructions** and **data**, a processing component with an **arithmetic logic unit (ALU)** and **registers**, and a **control unit** with an **instruction register** and a **program counter**.
- Many modern processors continue to implement this general structure, now known as the **von Neumann architecture**.
- Early applications of ENIAC included analyses related to the development of the **hydrogen bomb** and the computation of **firing tables for long-range artillery**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: ENIAC

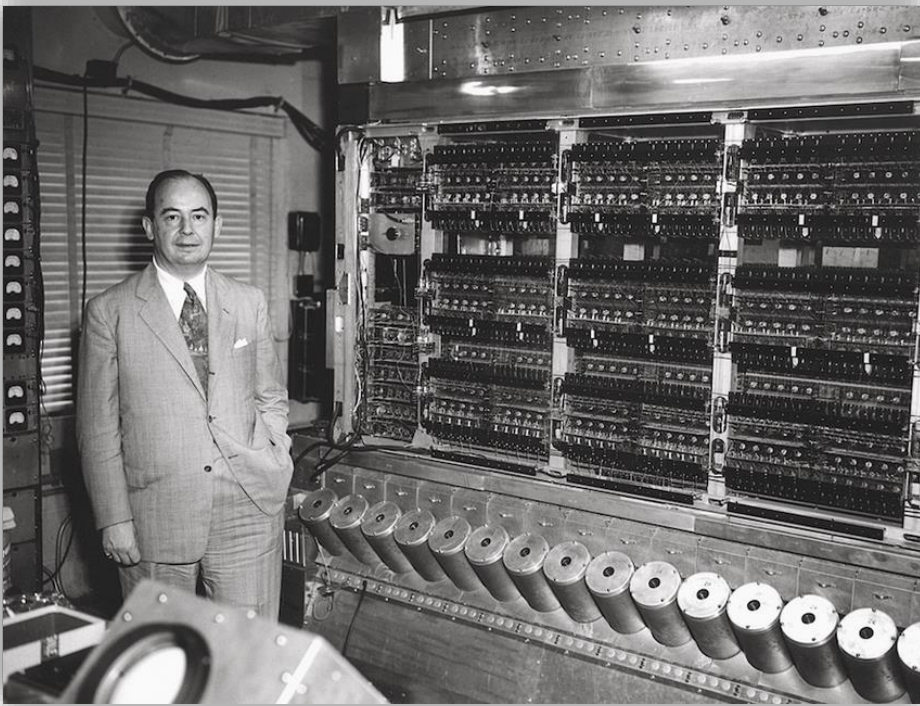


Figure 1.10: John Von Neumann

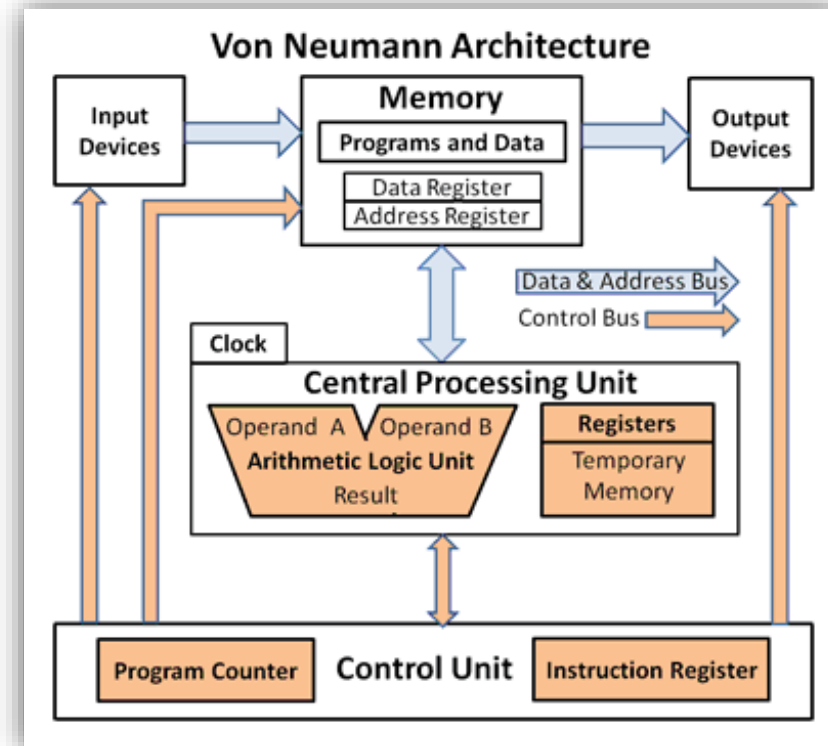


Figure 1.11: Von Neumann Architecture

- YouTube video (What is Von Neumann Architecture): <https://www.youtube.com/watch?v=-SADbPS8UgA>

— Computer Architecture

Chapter 1: Introducing Computer Architecture

Contents

1. Introduction
2. **The evolution of automated computing devices**
 - Charles Babbage's Analytical Engine
 - ENIAC
 - **IBM PC**
 - The iPhone
3. Moore's law
4. Computer architecture
 - The 6502 microprocessor
 - The 6502 instruction set

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC

- In the years following the construction of ENIAC, several technological breakthroughs resulted in **remarkable advances in computer architectures**:
 - The invention of the **transistor** in **1947** by **John Bardeen**, **Walter Brattain**, and **William Shockley** delivered a vast improvement over the vacuum tube technology prevalent at the time. Transistors were faster, smaller, consumed less power, and, once production processes had been sufficiently optimized, were much more reliable than the failure-prone tubes.
 - The commercialization of **integrated circuits** in **1958**, led by **Jack Kilby** of **Texas Instruments**, began the process of combining large numbers of formerly discrete components onto a single chip of silicon.
 - In **1971**, **Intel** began production of the first commercially available **microprocessor**, the **Intel 4004**. The 4004 was intended for use in **electronic calculators** and was specialized to operate on 4-bit binary coded decimal digits. From the humble beginning of the Intel 4004, microprocessor technology advanced rapidly over the ensuing decade by packing increasing numbers of circuit elements onto each chip and expanding the capabilities of the microprocessors implemented on those chips.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

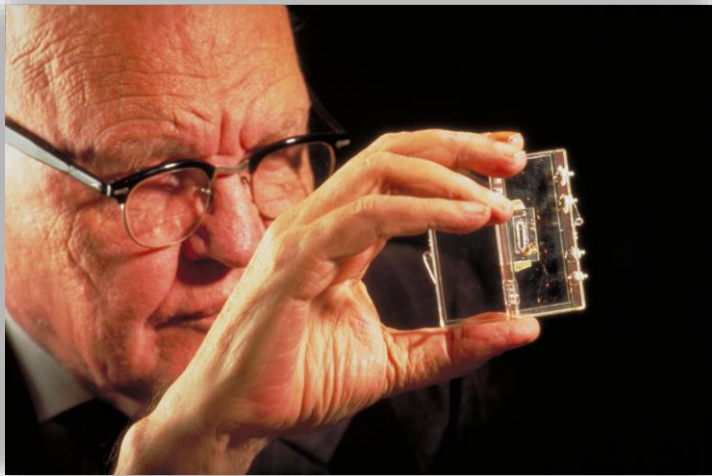


Figure 1.13: Jack Kilby – integrated circuit



Figure 1.12: John Bardeen, Walter Brattain & William Shockley

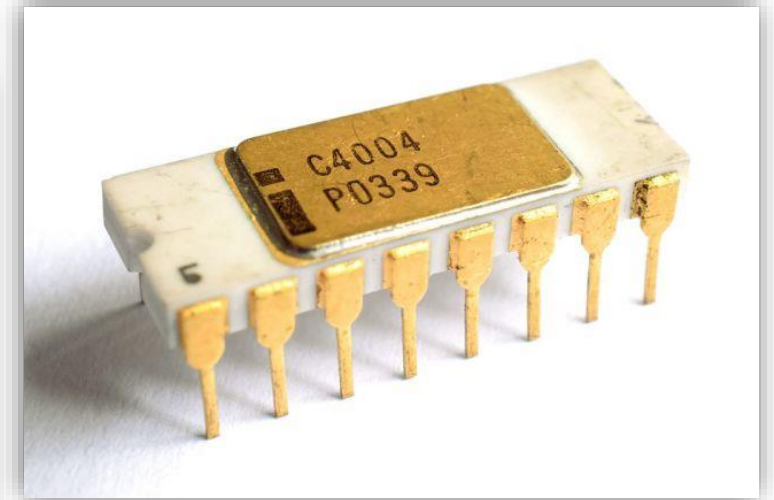


Figure 1.14: Intel 4004 microprocessor

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 8088 microprocessor

- IBM released the **IBM PC** in **1981**. The original PC contained an **Intel 8088 microprocessor** running at a **clock frequency** of **4.77 MHz** and featured **16 KB of RAM**, expandable to 256 KB.
- It included one or, optionally, two **floppy disk drives**. A **color monitor** was also available.
- Later versions of the PC supported **more memory**, but because portions of the address space had been reserved for video memory and read-only memory, the architecture could support a **maximum of 640 KB of RAM**.
- The 8088 contained **fourteen 16-bit registers**. Four were **general purpose registers** (AX, BX, CX, and DX.) Four were memory **segment registers** (CS, DS, SS, and ES) that extended the address space to **20 bits**.
- Segment addressing functioned by adding a 16-bit segment register value, shifted left by four bit positions, to a 16-bit offset contained in an instruction to produce a physical memory address within a **one megabyte range**.
- The **remaining 8088 registers** were the **Stack Pointer** (SP), the **Base Pointer** (BP), the **Source Index** (SI), the **Destination Index** (DI), the **Instruction Pointer** (IP), and the **Status Flags** (FLAGS).
- Modern x86 processors employ an architecture remarkably similar to this register set (*Chapter 10, Modern Processor Architectures and Instruction Sets*, will cover the details of the x86 architecture).
- The most obvious differences between the 8088 and **x86** are the extension of the register **widths to 32 bits in x86** and the **addition** of a pair of **segment registers (FS and GS)** that are used today primarily as **data pointers** in **multithreaded operating systems**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

Figure 1.15: IBM PC

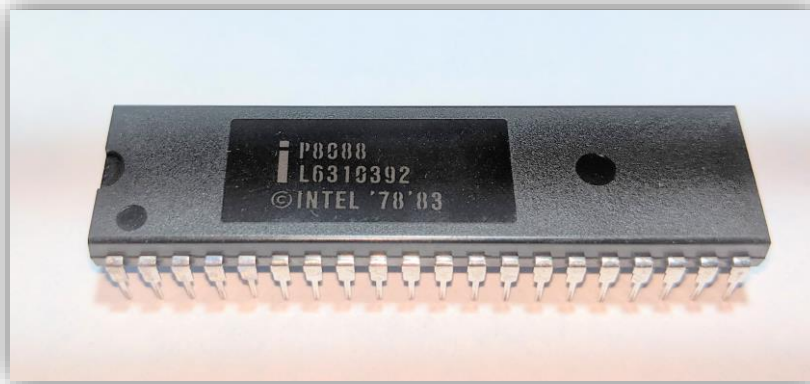


Figure 1.16: the Intel 8088 microprocessor

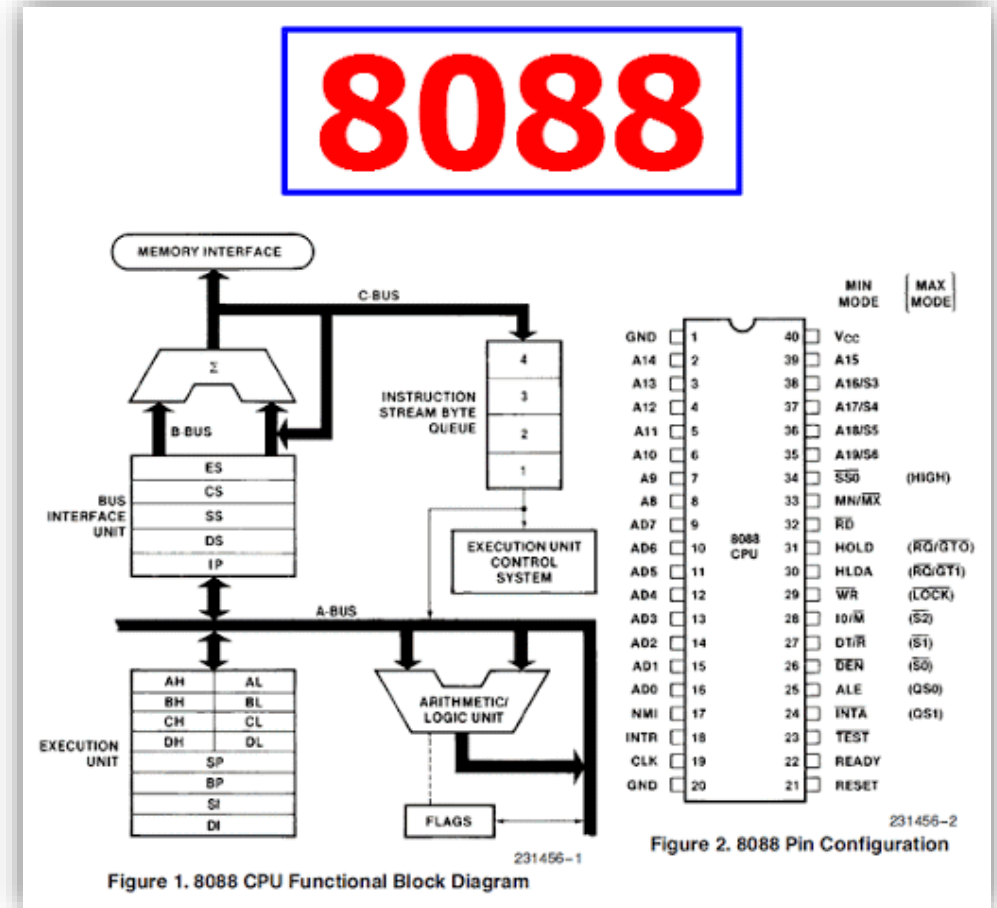


Figure 1.17: the Intel 8088 microprocessor

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 8088 microprocessor

- The **8088** had an external data bus width of **8 bits**, which meant it took two bus cycles to read or write a 16-bit value. This was a performance downgrade compared to the earlier **8086** processor, which employed a **16-bit external bus**.
- However, the use of the 8-bit bus made the PC more economical to produce and provided compatibility with lower-cost 8-bit peripheral devices. This cost-sensitive design approach helped to reduce the purchase price of the PC to a level accessible to more potential customers.
- **Program memory** and **data memory shared** the **same address space**, and the 8088 accessed memory over a single bus. In other words, the 8088 implemented the **von Neumann architecture**.
- The **8088 instruction set** included instructions for data movement, arithmetic, logical operations, string manipulation, control transfer (conditional and unconditional jumps and subroutine call and return), input/output, and additional miscellaneous functions.
- The processor required about **15 clock cycles per instruction on average**, resulting in an execution speed of **0.3 million instructions per second (MIPS)**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 8088 microprocessor

- The 8088 supported **nine** distinct **modes for addressing memory**. This variety of modes was needed to efficiently implement methods for accessing a single item at a time as well as for iterating through sequences of data.
- The segment registers in the 8088 architecture provided a clever way to expand the range of addressable memory without increasing the length of most instructions referencing memory locations.
- Each **segment** register allowed access to a **64-kilobyte block of memory** beginning at a physical memory address defined at a multiple of 16 bytes. In other words, the 16-bit segment register represented a **20-bit base address** with the lower four bits set to zero.
- Instructions could then reference any location within the **64-kilobyte segment** using a **16-bit offset** from the address defined by the segment register.
- The **CS register** selected the **code segment** location in memory and was used in fetching instructions and performing jumps and subroutine calls and returns.
- The **DS register** defined the **data segment** location for use by instructions involving the transfer of data to and from memory.
- The **SS register** set the **stack segment** location, which was used for local memory allocation within subroutines and for storing subroutine return addresses.

Computer Architecture

Chapter 1: Introducing Computer Architecture

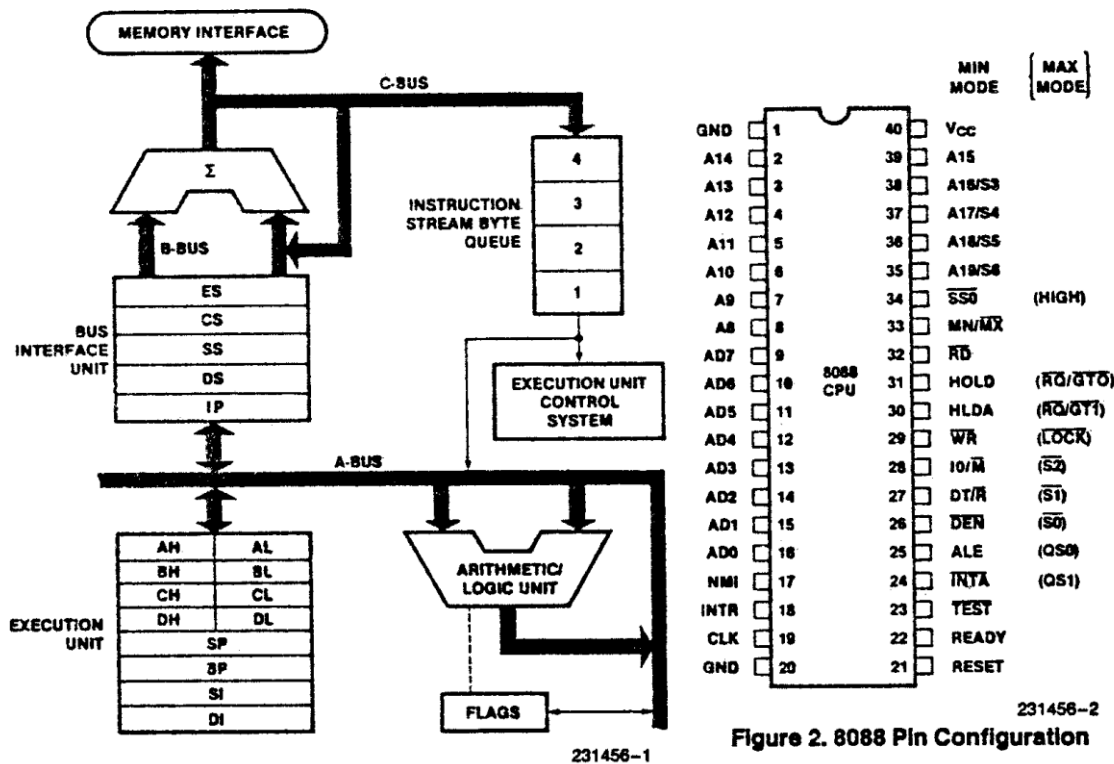


Figure 1. 8088 CPU Functional Block Diagram

Figure 2. 8088 Pin Configuration

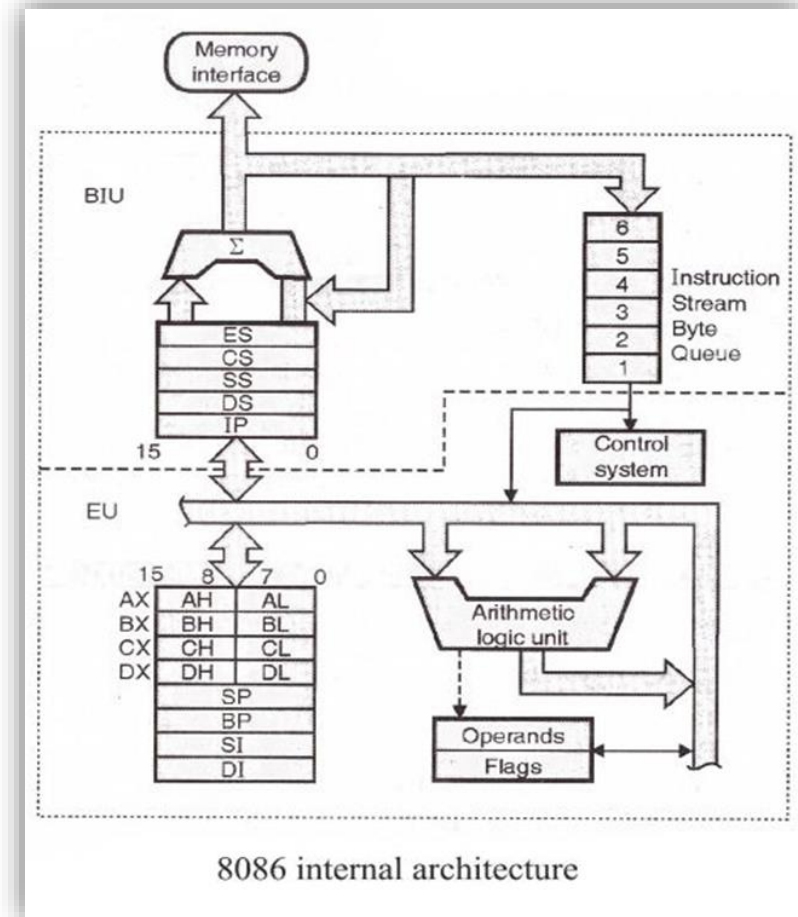


Figure 1.19: the Intel 8086 microprocessor

Figure 1.18: the Intel 8088 internal architecture

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 8088 microprocessor

- Programs that required less than 64-kilobyte in each of the code, data, and stack segments could ignore the segment registers entirely because those registers could be set once at program startup (compilers would do this automatically) and remain unchanged through execution.
- Things got quite a bit more complicated when a program's data size increased beyond 64-kilobyte.
- Compilers for the 8088 architecture distinguished between *near* and *far* references to memory.
- A **near pointer** represented a **16-bit offset** from the current segment register base address.
- A **far pointer** contained **32 bits** of addressing information: a **16-bit segment register** value and a **16-bit offset**.
- Far pointers obviously required 16 bits of extra data memory and they required additional processing time.
- Making a single memory access using a **far pointer** involved the following steps:
 1. Save the current segment register contents to a temporary location.
 2. Load the new segment value into the register.
 3. Access the data (read or write as needed) using an offset from the segment base.
 4. Restore the original segment register value.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 8088 microprocessor

- When using **far pointers**, it was possible to declare **data objects** (for example, an array of characters) **up to 64 KB** in size.
- If you needed a larger structure, you had to work out how to break it into chunks no larger than 64 KB and manage them yourself.
- As a result of these segment register manipulations, programs that required extensive access to data larger than 64 KB were susceptible to code size bloat and slower execution.
- The **IBM PC motherboard** also contained a socket for an **optional Intel 8087 floating point coprocessor**.
- The designers of the 8087 invented data formats and processing rules for **32-bit** and **64-bit** floating point numbers that became enshrined in 1985 as the **IEEE 754 floating-point standard**, which remains in near-universal use today.
- The **8087** could perform about **50,000 floating-point operations per second**.
- We will look at **floating-point processors** in detail in *Chapter 9, Specialized Processor Extensions*.

Computer Architecture

Chapter 1: Introducing Computer Architecture

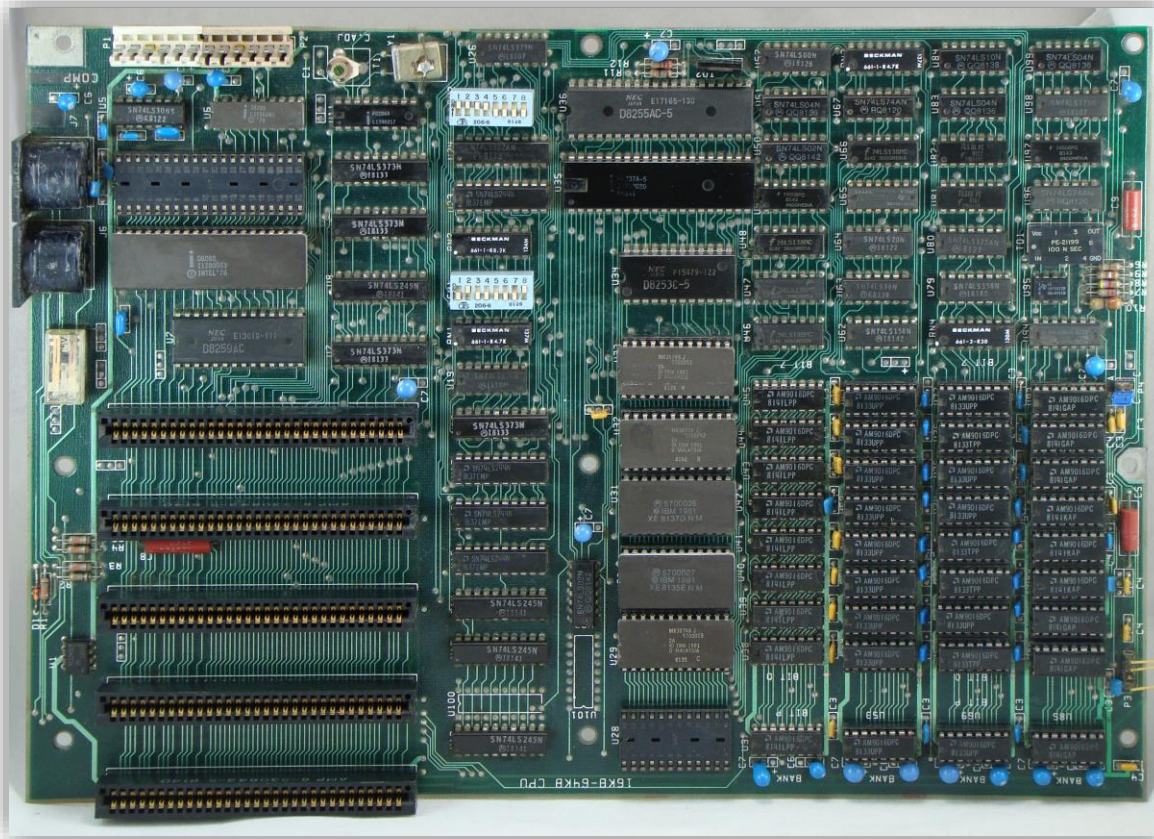


Figure 1.20: original IBM PC motherboard

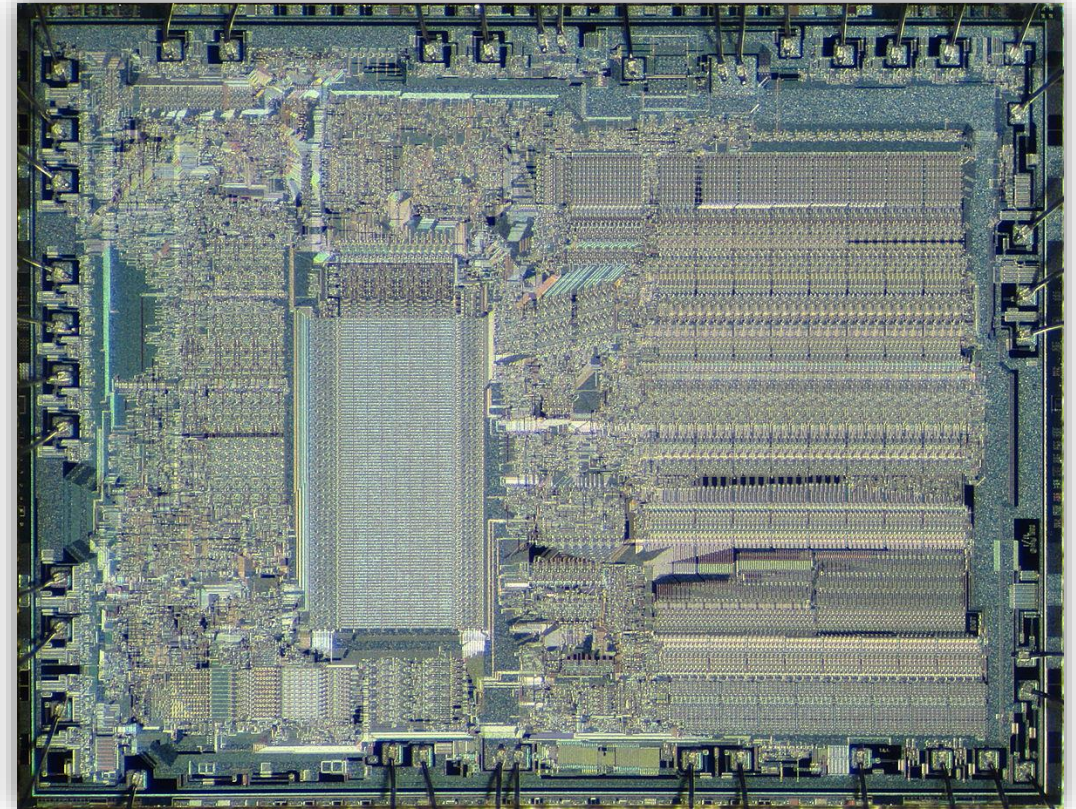


Figure 1.21: Intel 8087 coprocessor die

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 80286 & 80386 microprocessors

- The second generation of the IBM PC, the **PC AT**, was released in 1984.
- **AT** stood for **Advanced Technology** and referred to several significant enhancements over the original PC that mostly resulted from the use of the **Intel 80286 processor**.
- Like the 8088, the 80286 was a **16-bit processor**, and it maintained backward compatibility with the 8088: 8088 code could run unmodified on the 80286.
- The **80286** had a **16-bit data bus** and **24 address lines** supporting a **16-megabyte address space**.
- The external **data bus width** was **16 bits**, improving data access performance over the 8-bit bus of the 8088.
- The **instruction execution rate** (instructions per clock cycle) was about **double** the 8088 in many applications. This meant that at the same clock speed the 80286 would be twice as fast as the 8088.
- The original PC AT clocked the processor at **6 MHz** and a *later* version operated at **8 MHz**.
- The **6 MHz variant** of the 80286 achieved an instruction execution rate of about **0.9 MIPS**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

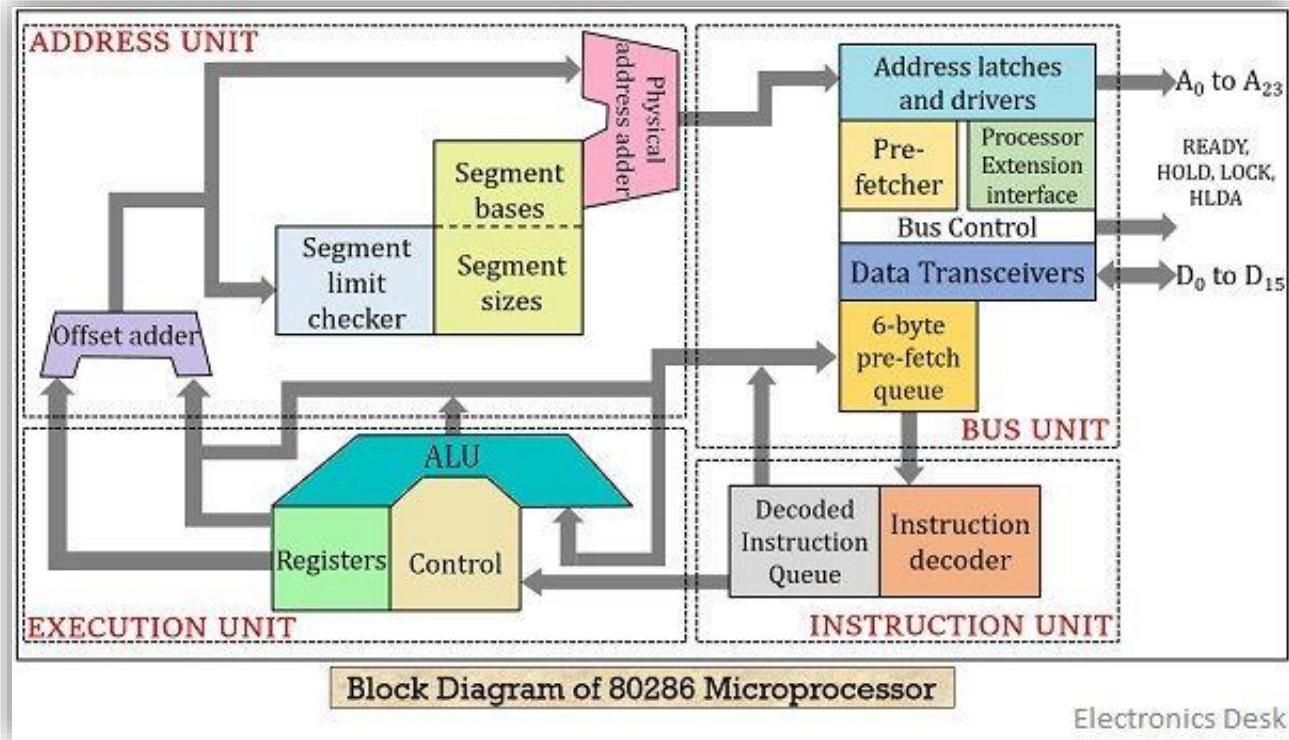


Figure 1.22: Intel 80286 microprocessor architecture

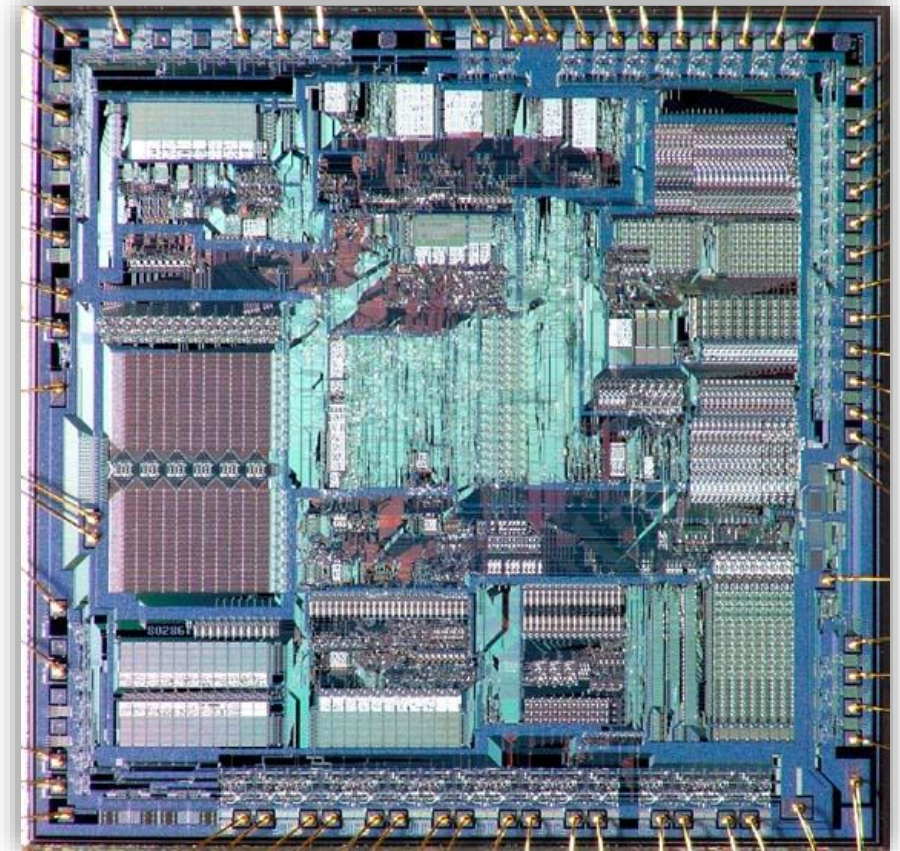


Figure 1.23: Intel 80286 coprocessor die

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 80286 & 80386 microprocessors

- The **80286** implemented a **protected virtual address mode** intended to support multiuser operating systems and multitasking.
- In **protected mode**, the processor enforced **memory protection** to ensure one user's programs could not interfere with the operating system or with other users' programs.
- This groundbreaking technological advance remained little used for many years, mainly because of the prohibitive cost of adding sufficient memory to a computer system to make it useful in a multiuser, multitasking context.
- The next generation of the x86 processor line was the **80386**, introduced in **1985**. The 80386 was a **32-bit processor** with support for a **flat 32-bit memory model** in protected mode.
- The **flat memory model** allowed programmers to address up to **4 GB directly**, without the need to manipulate segment registers.
- **Compaq** introduced an IBM PC-compatible personal computer based on the 80386 called the **DeskPro in 1986**. The DeskPro shipped with a version of Microsoft Windows targeted to the 80386 architecture.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

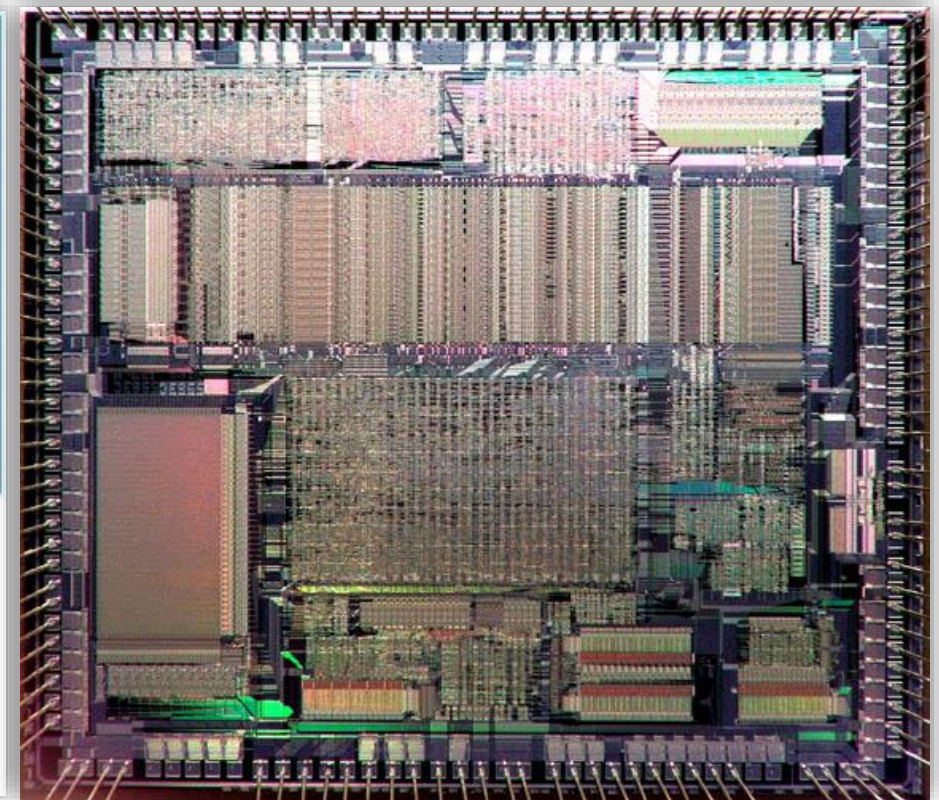
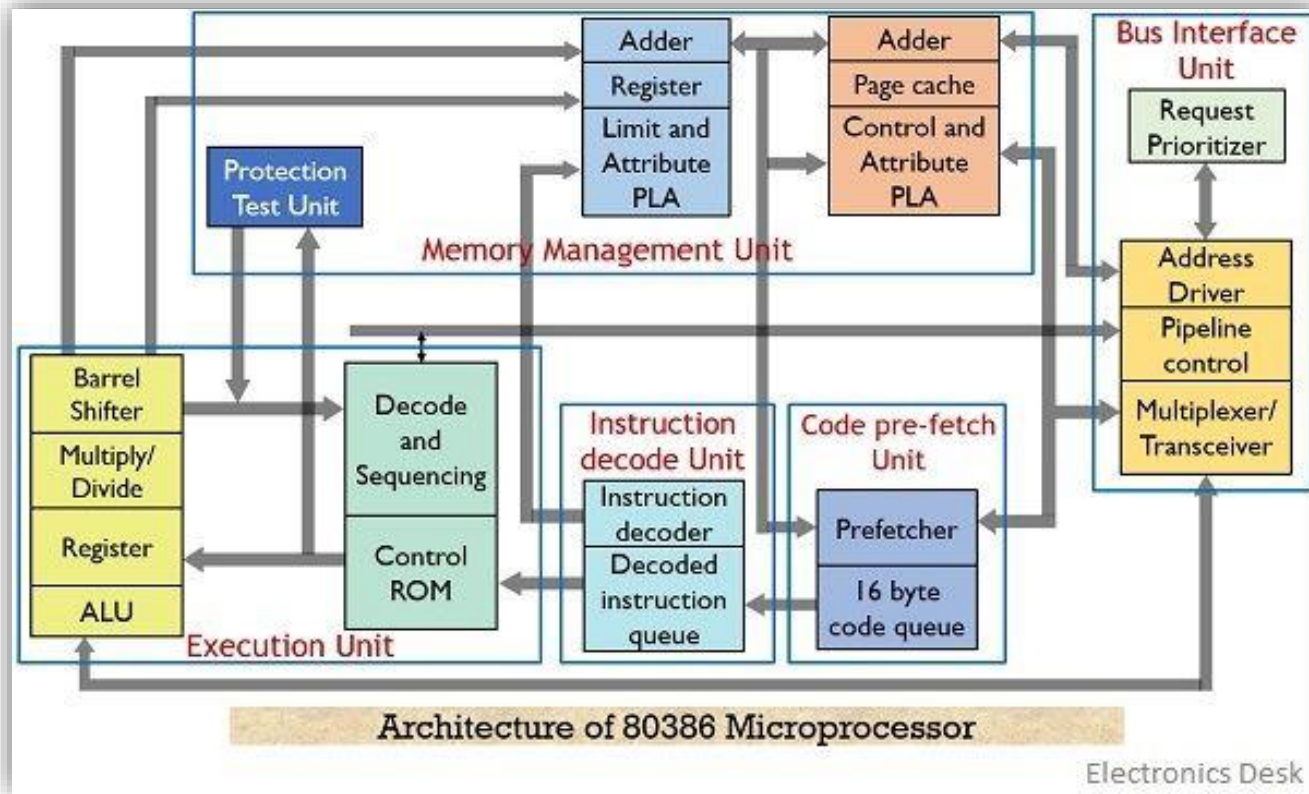


Figure 1.24: Intel 80386 microprocessor architecture

Figure 1.25: Intel 80386 coprocessor die

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The 80286 & 80386 microprocessors

- The 80386 maintained a large degree of backward compatibility with the 80286 and 8088 processors.
- The design implemented in the 80386 remains the current standard x86 architecture.
- Much more about this architecture will be covered in *Chapter 10, Modern Processor Architectures and Instruction Sets*.
- The initial version of the **80386** was clocked at **33 MHz** and achieved about **11.4 MIPS**.
- Modern implementations of the x86 architecture run several hundred times faster than the original as the result of **higher clock speeds**, performance enhancements such as extensive use of **cache memory**, and **more efficient instruction execution** at the hardware level.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

1.1 BRIEF HISTORY OF THE x86 FAMILY 80286, 80386, and 80486

Table 1-2: Evolution of Intel's Microprocessors (from the 8086 to the Pentium Pro)

Product	8086	80286	80386	80486	Pentium	Pentium Pro
Year Introduced	1978	1982	1985	1989	1993	1995
Technology	NMOS	NMOS	CMOS	CMOS	BICMOS	BICMOS
Clock rate (MHz)	3–10	10–16	16–33	25–33	60, 66	150
Number of pins	40	68	132	168	273	387
Number of transistors	29,000	134,000	275,000	1.2 mill.	3.1 mill.	5.5 mill.
Physical memory	1M	16M	4G	4G	4G	64G
Virtual memory	None	1G	64T	64T	64T	64T
Internal data bus	16	16	32	32	32	32
External data bus	16	16	32	32	64	64
Address bus	20	24	32	32	32	36
Data types	8/16	8/16	8/16/32	8/16/32	8/16/32	8/16/32

PEARSON

The x86 PC
Assembly Language, Design, and Interfacing
By Muhammad Ali Mazidi, Janice Gillespie, Mazidi and Danny Causey

© 2010, 2003, 2000, 1998 Pearson Higher Education, Inc.
Pearson Prentice Hall - Upper Saddle River, NJ 07458

Figure 1.26: brief history of the x86 Intel microprocessor family

— **Computer Architecture**

Chapter 1: Introducing Computer Architecture

Contents

1. Introduction
- 2. The evolution of automated computing devices**
 - Charles Babbage's Analytical Engine
 - ENIAC
 - IBM PC
 - **The iPhone**
3. Moore's law
4. Computer architecture
 - The 6502 microprocessor
 - The 6502 instruction set

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The iPhone

- In **2007**, Steve Jobs introduced the **iPhone** to a world that had no idea it had any use for such a device.
- The iPhone built upon previous revolutionary advances from **Apple Computer** including the **Macintosh computer** in 1984 and the iPod music player in 2001.
- The **iPhone** combined the functions of the **iPod**, a **mobile telephone**, and an **Internet-connected computer**.
- The iPhone did away with the hardware keyboard that was common on smartphones of the time and replaced it with a **touchscreen** capable of displaying an on-screen keyboard or any other type of user interface.
- The screen was driven by the user's fingers and supported **multi-finger gestures** for actions such as zooming a photo.
- The iPhone ran the **OS X operating system**, the same OS used on the flagship Macintosh computers of the time.
- This decision immediately enabled the iPhone to support a **vast range of applications** already developed for Macs and empowered software developers to rapidly introduce new applications tailored to the iPhone, once Apple began allowing third-party application development.

— Computer Architecture

Chapter 1: Introducing Computer Architecture



Figure 1.27: Steve Jobs (Apple) announcing the iPhone



Figure 1.28: the iPhone

— Computer Architecture

Chapter 1: Introducing Computer Architecture

2. The evolution of automated computing devices: IBM PC – The iPhone

- The iPhone 1 had a 3.5" screen with a resolution of 320x480 pixels. It was 0.46 inches thick (thinner than other smartphones), had a 2-megapixel camera built in, and weighed 4.8 oz.
- A **proximity sensor** detected when the phone was held to the user's ear and turned off screen illumination and touchscreen sensing during calls. It had an **ambient light sensor** to automatically set the screen brightness and an **accelerometer** detected whether the screen was being held in portrait or landscape orientation.
- The iPhone 1 included 128 MB of RAM, 4 GB, 8 GB, or 16 GB of flash memory, and supported Global System for Mobile communications (**GSM**) cellular communication, **Wi-Fi** (802.11b/g), and **Bluetooth**.
- The iPhone 1 processor was a **32-bit ARM11** manufactured by **Samsung** running at 412 MHz. The ARM11 was an improved variant of previous generation ARM processors and included an 8-stage instruction pipeline and support for **Single Instruction-Multiple Data (SIMD)** processing to improve audio and video performance.
- The ARM processor architecture will be discussed further in *Chapter 10, Modern Processor Architectures and Instruction Sets*.
- The iPhone 1 was powered by a **3.7V lithium-ion polymer battery**. The battery was not intended to be replaceable, and Apple estimated it would lose about 20 percent of its original capacity after 400 charge and discharge cycles. Apple quoted up to 250 hours of standby time and 8 hours of talk time on a single charge.
- Six months after the iPhone was introduced, Time magazine named the iPhone the "**Invention of the Year**" for 2007. In 2017, Time ranked the 50 Most Influential Gadgets of All Time. The iPhone topped the list.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

Contents

1. Introduction
2. The evolution of automated computing devices
 - Charles Babbage's Analytical Engine
 - ENIAC
 - IBM PC
 - The iPhone
- 3. Moore's law**
4. Computer architecture
 - The 6502 microprocessor
 - The 6502 instruction set

— Computer Architecture

Chapter 1: Introducing Computer Architecture

3. Moore's law

- **Gordon Moore** co-founded **Fairchild Semiconductor** in **1957** and was later the chairman and CEO of **Intel**.
- In **1965**, Moore published an article in *Electronics magazine* in which he offered his prediction of the changes that would occur in the semiconductor industry over the following ten years.
- In the article, he observed that the number of formerly discrete components such as transistors, diodes, and capacitors that could be integrated onto a single chip had been **doubling approximately yearly** and the trend was likely to continue over the subsequent ten years.
- *This doubling formula came to be known as Moore's law.*
- This was **not a scientific law** in the sense of the law of gravity. Rather, it was based on observation of historical trends, and he believed this formulation had some ability to predict the future. Moore's law turned out to be impressively accurate over those ten years.
- In **1975**, he revised the predicted growth rate for the following ten years to doubling the number of components per integrated circuit **every two years rather than yearly**. This pace continued for decades, up until about 2010.
- In more recent years, the growth rate has appeared to decline slightly.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

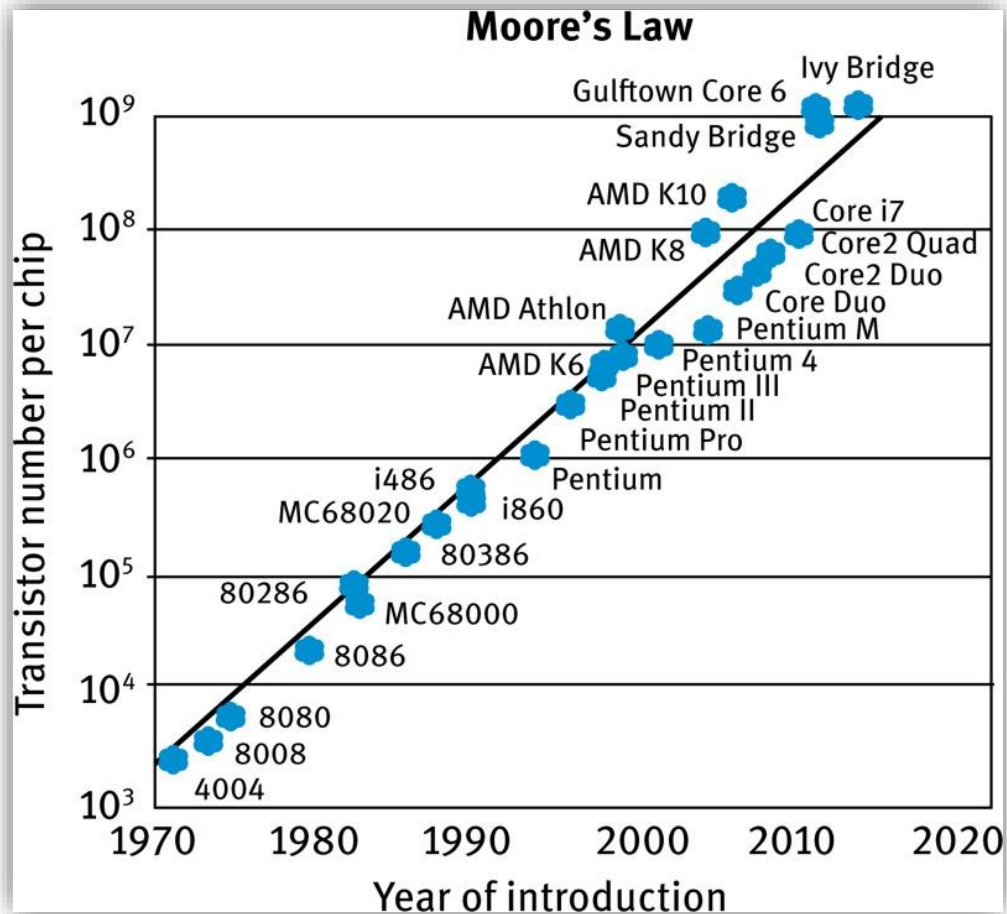


Figure 1.29: Moore's law

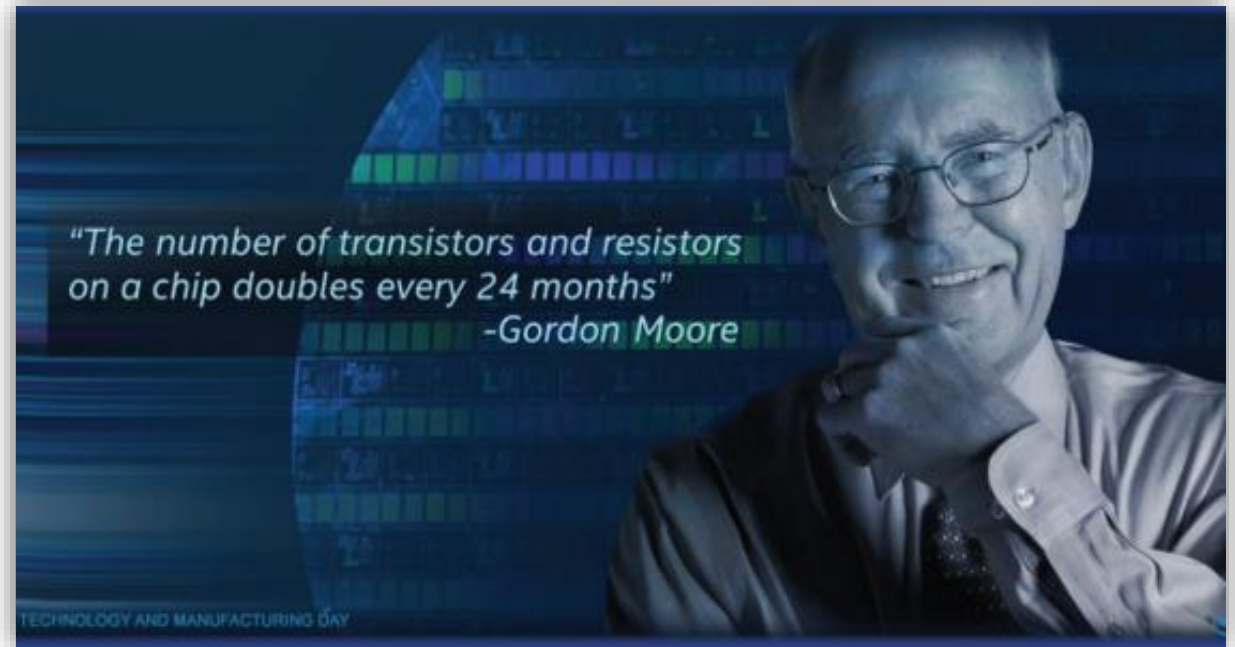


Figure 1.30: Gordon Moore

— Computer Architecture

Chapter 1: Introducing Computer Architecture

3. Moore's law

- Moore's law has proven to be a reliable tool for evaluating the **performance of semiconductor companies** over the decades.
- Companies have used it to set **goals** for the **performance** of their **products** and to **plan** their **investments**. By comparing the integrated circuit density increases for a company's products against prior performance, and against other companies, it is possible for semiconductor executives and industry analysts to evaluate and score company performance.
- The results of these analyses have fed directly into decisions **to build enormous new fabrication plants** and to **push the boundaries of ever-smaller integrated circuit feature sizes**.
- The decades since the introduction of the IBM PC have seen tremendous growth in the capability of single-chip microprocessors.
- Current processor generations are hundreds of times faster, operate on 32-bit and 64-bit data natively, have far **more integrated memory resources**, and unleash vastly **more functionality**, all packed into a single integrated circuit.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

3. Moore's law

- The increasing density of semiconductor features, as predicted by Moore's law, has enabled all of these improvements. **Smaller transistors** run at **higher clock speeds** due to the **shorter connection paths** between circuit elements.
- Smaller transistors also, obviously, allow **more functionality to be packed** into a given amount of die area.
- Being smaller and closer to neighboring components allows the transistors **to consume less power** and **generate less heat**.
- Traditional semiconductor manufacturing processes have begun to approach **physical limits** that will eventually put the brakes on growth under Moore's law.
- The smallest features on current commercially available integrated circuits are around **5 nanometers** (nm).
- For comparison, a typical human hair is about 50,000 nm thick and a **water molecule** (one of the smallest molecules) is **0.28 nm across**.
- There is a point beyond which it is simply not possible for circuit elements to become smaller as the sizes approach atomic scale.
- One approach is **specialization**, in which circuits are designed to perform a specific category of tasks extremely well rather than performing a wide variety of tasks merely adequately.

Computer Architecture

Chapter 1: Introducing Computer Architecture

Limits	Engineering	Design and Validation	Energy, time	Space, time	Information, Complexity
Funda-mental	Abbe (diffraction) Amdahl Gustafson	Error-corr. & dense codes Fault-tolerance thresholds	Einstein $E=mc^2$ Heisenberg $\Delta E \Delta t$ Landauer $kT \ln 2$ Bremermann Adiabatic thrms	Speed of light Planck scale Bekenstein Fisher $T(n)^{1/(d+1)}$	Shannon Holevo NC, NP, #P Turing (decidability)
Material	Dielectric constant Carrier mobility Surface morphology Fabrication-related	Analytical & numerical modeling	Conductivity Permittivity Bandgap Heat flow	Propagation speed Atomic spacing No gravitational collapse	Information transfer between carriers
Device	Gate dielectric Channel charge ctrl Leakage, Latency Crosstalk, Aging	Compact modeling Parameter selection	CMOS, quantum Charge-centric Signal to noise Energy conversion	Entropy density Entropy flow Interfaces & contacts Size & delay variation	Universality
Circuit	Delay, Inductance Thermal-related Yield, Reliability, IO	Interconnect Test Validation	Dark, darker, dim and gray silicon Cooling efficiency Power density/supply	Interconnect 2D or 3D	Circuit complexity bounds
System +SW	Specification, Implementation Validation, Cost		Synchronization, Physical integration Parallelism, <i>Ab initio</i> limits (Lloyd)		The CAP theorem

Figure 1.31: limits to Moore's law

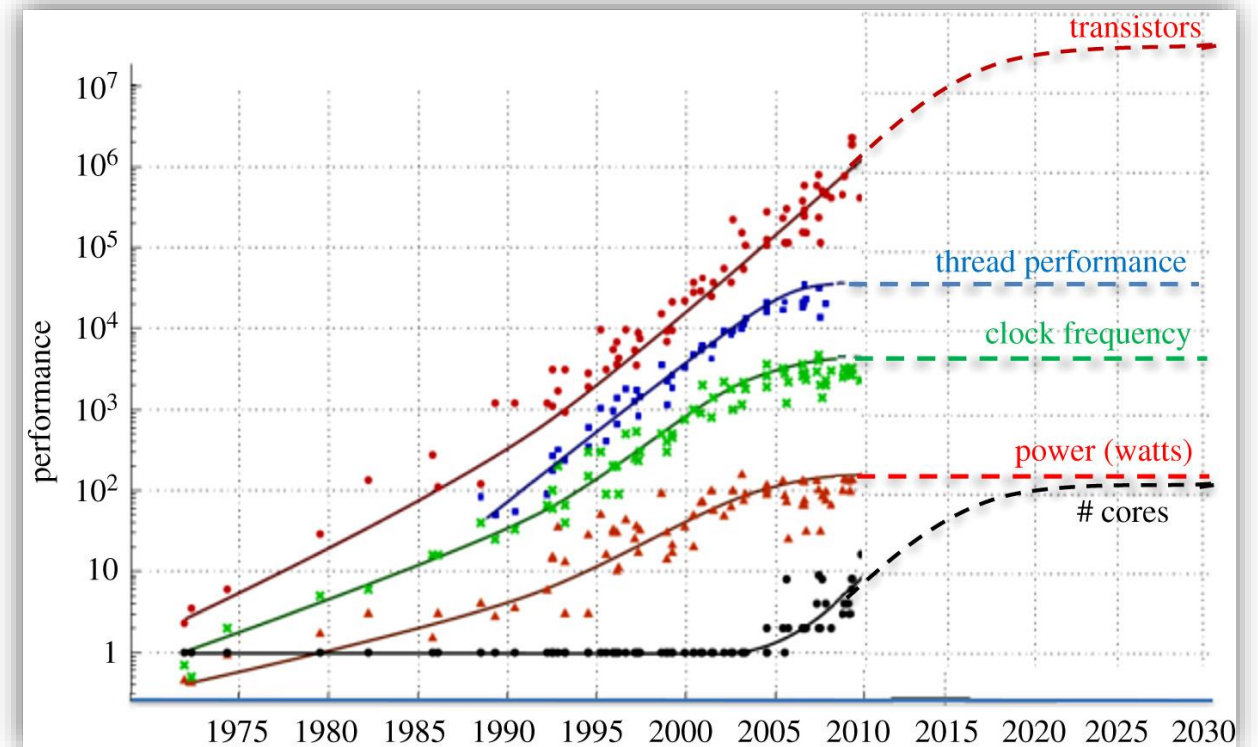


Figure 1.32: beyond Moore's law

— Computer Architecture

Chapter 1: Introducing Computer Architecture

3. Moore's law

- Graphical Processing Units (GPUs) are an excellent example of **specialization**.
- Original GPUs focused exclusively on **improving** the **speed** at which **three-dimensional graphics scenes** could be **rendered**, mostly for use in **video gaming**.
- The calculations involved in generating a three-dimensional scene are well defined and must be applied to thousands of pixels to create a single frame.
- The process must be repeated for each subsequent frame, and frames may need to be redrawn at a 60 Hz or higher rate to provide a satisfactory user experience. The computationally demanding and repetitive nature of this task is ideally suited for **acceleration via hardware parallelism**.
- Multiple computing units within a GPU simultaneously perform essentially the same calculations on different input data to produce separate outputs. Those outputs are combined to generate the final scene.
- Modern GPU designs have been enhanced to support other domains, such as **training neural networks** on massive amounts of data.
- **GPUs** will be covered in detail in *Chapter 6, Specialized Computing Domains*.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

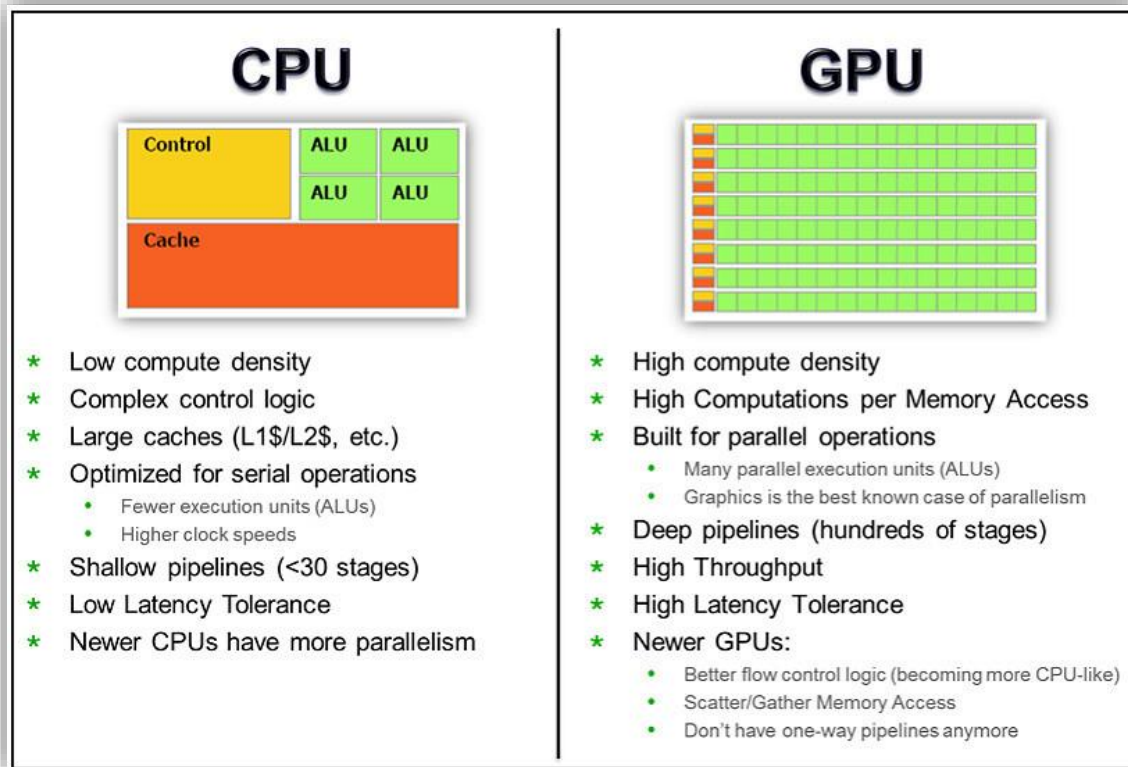


Figure 1.33: CPU/GPU comparison

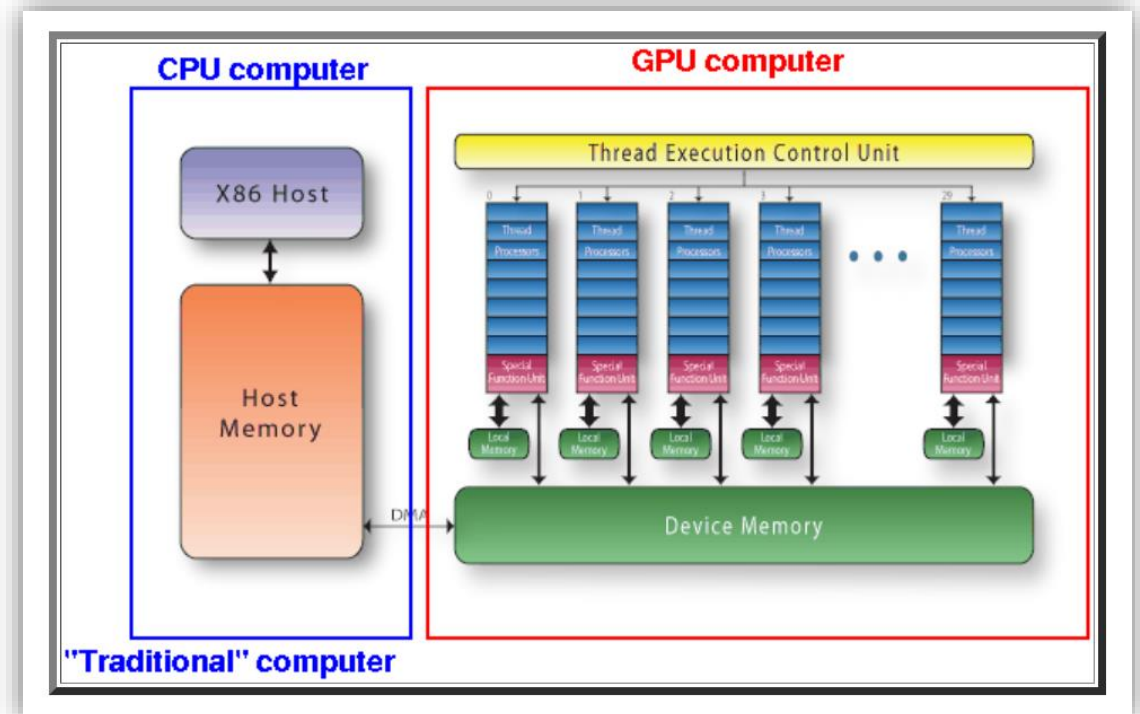


Figure 1.34: CPU computer versus GPU computer

— Computer Architecture

Chapter 1: Introducing Computer Architecture

3. Moore's law

- *As Moore's law shows signs of beginning to fade over the coming years, what advances might take its place to kick off the next round of innovations in computer architectures?*
- **Quantum computing** is one example of future technologies. We will cover that technology in *Chapter 17, Quantum Computing and Other Future Directions in Computer Architectures*.
- Quantum computing takes advantage of the properties of subatomic particles to perform computations in a manner that traditional computers cannot.
- A basic element of quantum computing is the **qubit**, or **quantum bit**. A qubit is similar to a regular binary bit, but in addition to representing the states 0 and 1, qubits can attain a state that is a superposition of the 0 and 1 states.
- When measured, the qubit output will always be 0 or 1, but the **probability** of producing either output is a function of the **qubit's quantum state prior to being read**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

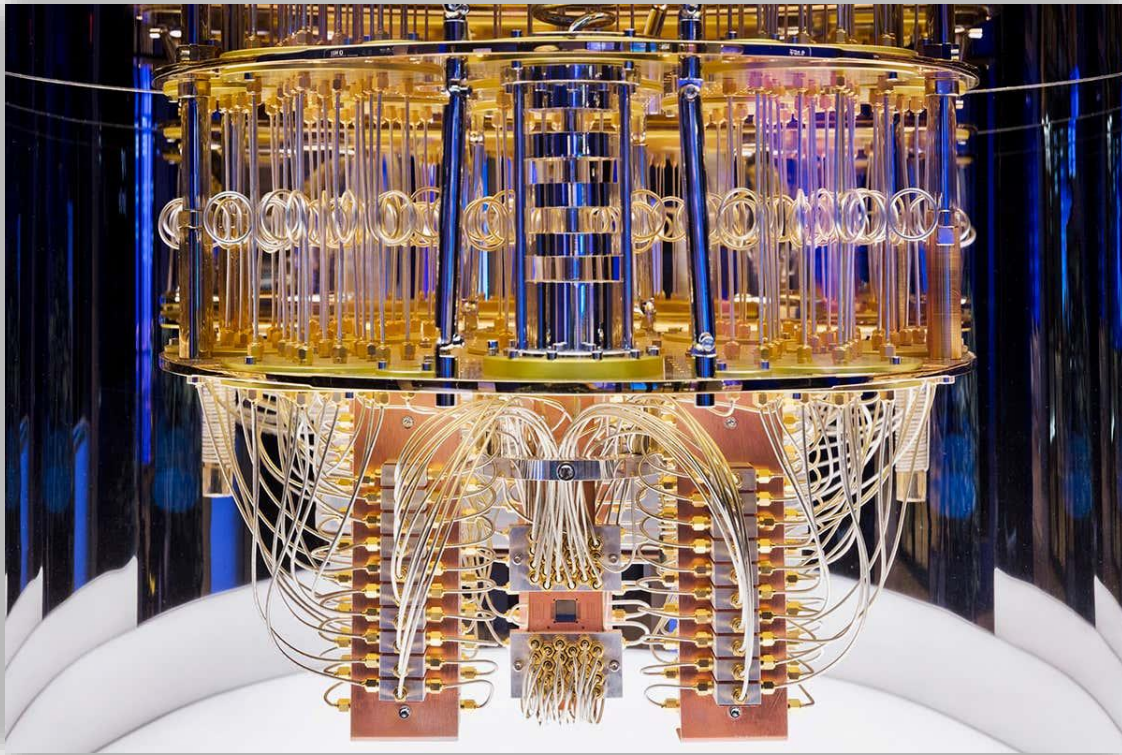


Figure 1.35: quantum computer

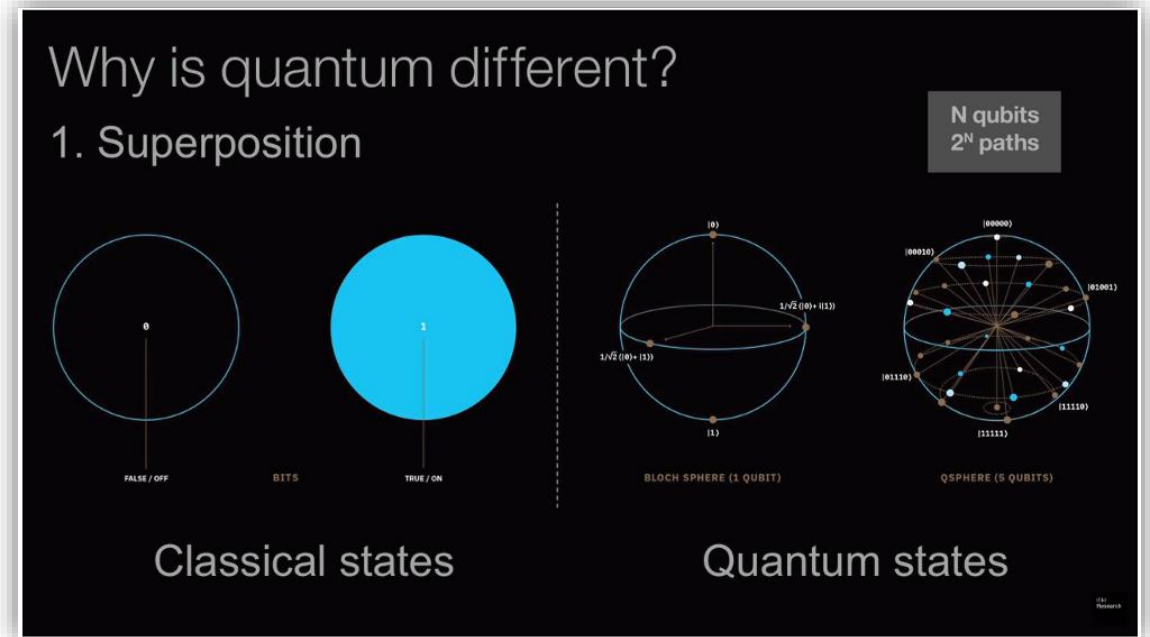


Figure 1.36: superposition of qubits

— **Computer Architecture**

Chapter 1: Introducing Computer Architecture

Contents

1. Introduction
2. The evolution of automated computing devices
 - Charles Babbage's Analytical Engine
 - ENIAC
 - IBM PC
 - The iPhone
3. Moore's law
- 4. Computer architecture**
 - **The 6502 microprocessor**
 - The 6502 instruction set

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 microprocessor

- The **6502** processor was introduced by **MOS Technology** in **1975**. The 6502 found widespread use in its early years in video game consoles from **Atari** and **Nintendo** and in computers marketed by **Commodore** and **Apple**.
- The 6502 continues in widespread use today in embedded systems, with estimates of between five and **ten billion units** produced as of **2018**.
- Many early microprocessors, like the 6502, were powered by a constant voltage of **5 Volts (5V)**. In these circuits, a *low* signal level is any voltage between 0 and 0.8V. A *high* signal level is any voltage between 2 and 5V. The low signal level is defined as logical 0 and the high signal level is defined as logical 1.
- The **word length** of a processor defines the size of the fundamental data element the processor operates upon. The 6502 has a word length of **8 bits**. This means the 6502 reads and writes memory 8 bits at a time and stores data internally in 8-bit wide registers.
- Program memory and data memory share the **same address space** and the 6502 accesses its memory over a single bus. As was the case with the Intel 8088, the 6502 implements the **von Neumann architecture**. The 6502 has a **16-bit address bus**, enabling access to **64 KB of memory**.

— Computer Architecture

Chapter 1: Introducing Computer Architecture



Figure 1.37: 6502 products – The BBC Micro



Figure 1.38: 6502 products – Atari 2600

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 microprocessor

- The 6502 contains internal storage areas called **registers**.
- A typical processor contains a small number of registers for temporarily storing data values and performing operations such as addition or address computation.
- The following **figure 1.39** shows the 6502 register structure.
- The processor contains five 8-bit registers (A, X, Y, P, and S) and one 16-bit register (PC).
- The numbers above each register indicate the bit numbers at each end of the register.
- Each of the A, X, and Y registers can serve as a general-purpose storage location.
- Program instructions can load a value into one of those registers and, some instructions later, use the saved value for some purpose, as long as the intervening instructions did not modify the register contents.
- The **A register** is the only register capable of performing **arithmetic** operations.
- The **X and Y registers**, but not the A register, can be used as **index registers** in calculating **memory addresses**.

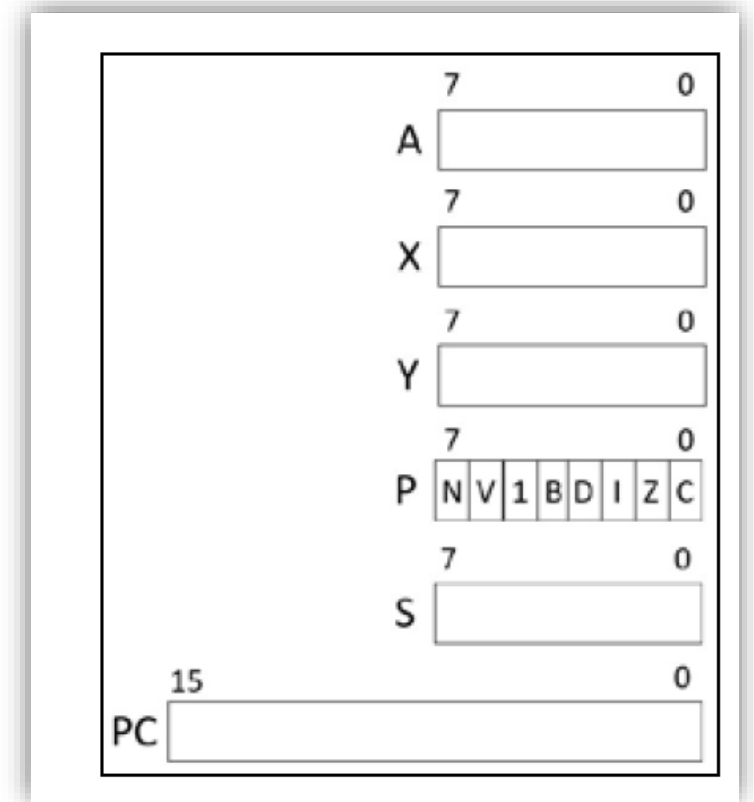


Figure 1.39: overview of 6502 registers

— Computer Architecture

Chapter 1: Introducing Computer Architecture



Figure 1.40: 6502 products – The Commodore 64



Figure 1.41: 6502 products – Apple II

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 microprocessor

- The **P register** contains **processor flags**.
- Each bit in this register has a unique purpose, except for the bit labeled 1. The 1 bit is unused and can be ignored.
- Each of the remaining bits in this register is called a **flag** and indicates a specific condition that has occurred or represents a configuration setting.
- The 6502 flags are as follows:
 - **N: Negative sign** flag: This flag is set when the result of an arithmetic operation sets bit 7 in the result. This flag is used in signed arithmetic.
 - **V: Overflow** flag: This flag is set when a signed addition or subtraction results in *overflow* or *underflow* outside the range -128 to 127.
 - **B: Break** flag: This flag indicates a Break (BRK) instruction has executed. This bit is not present in the P register itself. The B flag value is only relevant when examining the P register contents as stored onto the stack by a BRK instruction or by an interrupt. The B flag is set to distinguish a *software* interrupt resulting from a BRK instruction from a *hardware* interrupt during interrupt processing.
 - **D: Decimal mode** flag: This flag indicates processor arithmetic will operate in Binary-Coded Decimal (BCD) mode.

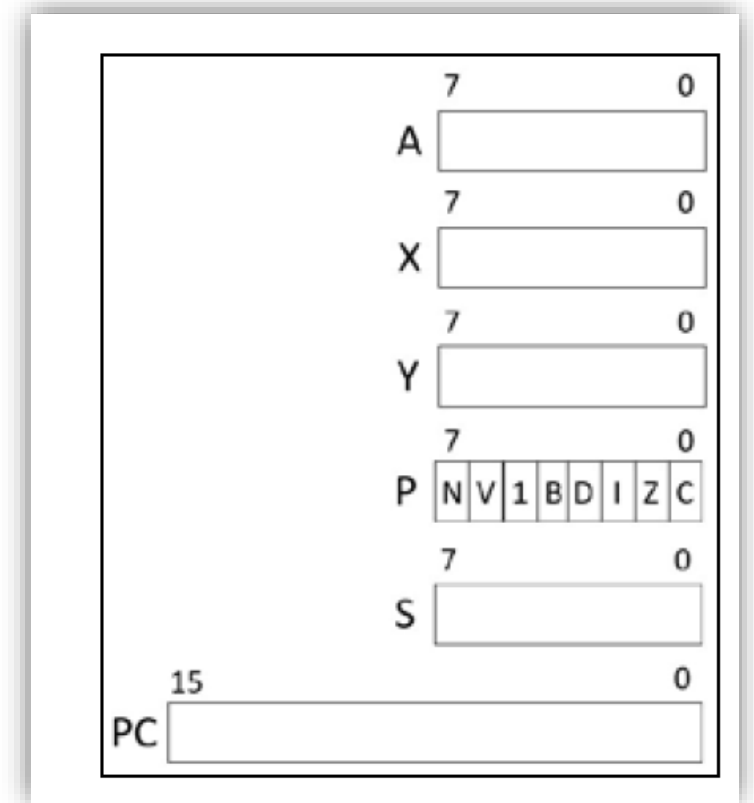


Figure 1.39: overview of 6502 registers

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 microprocessor

- The **P register** contains **processor flags**.
- Each bit in this register has a unique purpose, except for the bit labeled 1. The 1 bit is unused and can be ignored.
- Each of the remaining bits in this register is called a **flag** and indicates a specific condition that has occurred or represents a configuration setting.
- The 6502 flags are as follows:
 - **I: Interrupt disable** flag: This flag indicates that interrupt inputs (other than the non-maskable interrupt) will not be processed.
 - **Z: Zero** flag: This flag is set when an operation produces a result of zero.
 - **C: Carry** flag: This flag is set when an arithmetic operation produces a carry.

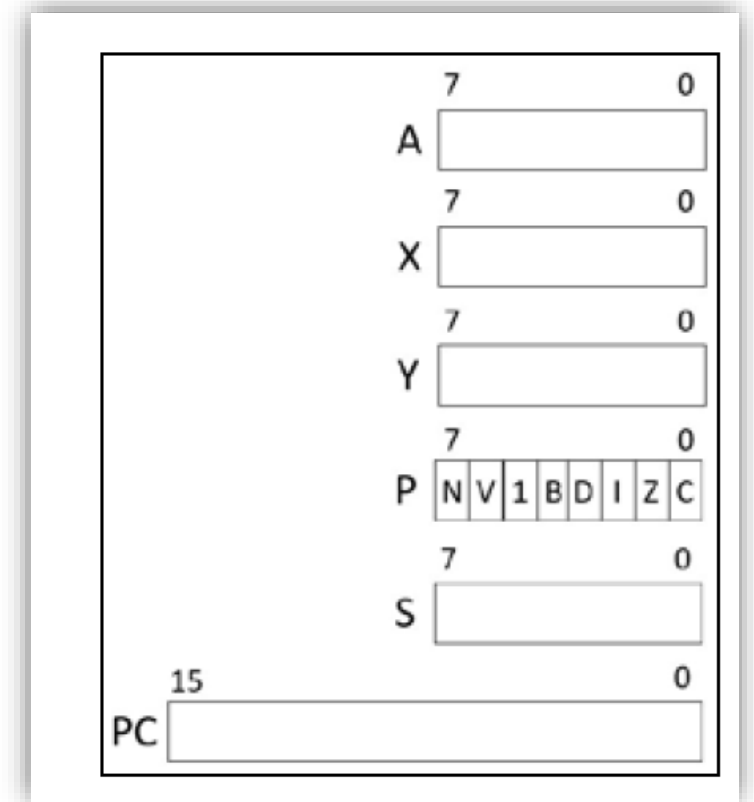


Figure 1.39: overview of 6502 registers

— Computer Architecture

Chapter 1: Introducing Computer Architecture



Figure 1.42: Nintendo Entertainment System

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 microprocessor

- The **S register** is the **stack pointer**. In the 6502, the stack is the region of memory from addresses \$100 to \$1FF. This **256-byte range** is used for temporary storage of parameters within subroutines and holds the return address when a subroutine is called.
- At **system startup**, the S register is initialized to point to the **top** of this range. Values are "pushed" onto the stack using instructions such as **PHA**, which pushes the contents of the A register onto the stack. When a value is pushed onto the stack, the 6502 stores the value at the address indicated by the S register, after adding the fixed \$100 offset, then decrements the S register.
- Additional values can be placed on the stack by executing more push instructions. As additional values are pushed, the stack grows **downward** in memory.

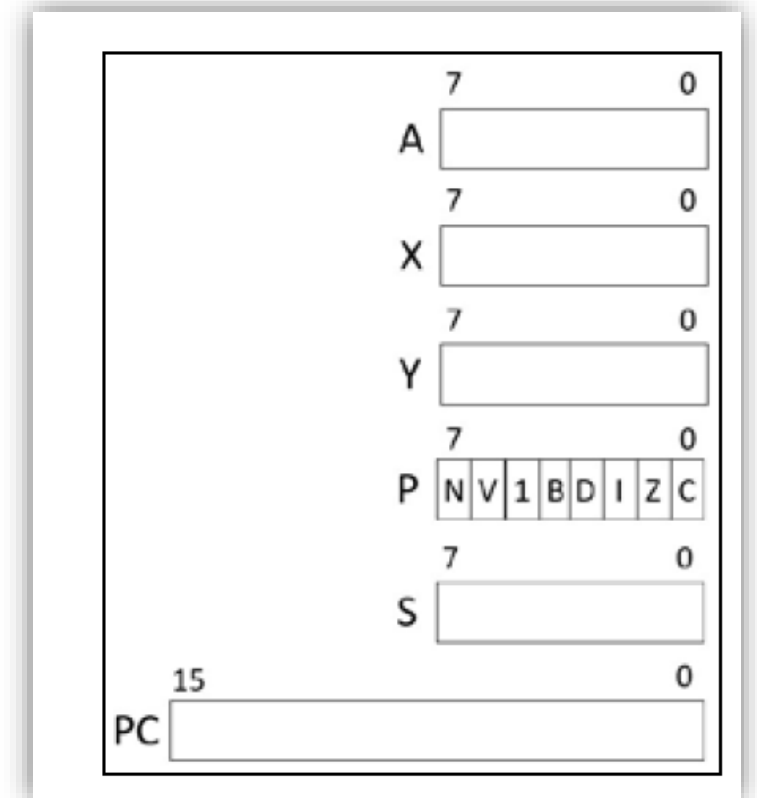


Figure 1.39: overview of 6502 registers

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 microprocessor

- Programs must take care not to exceed the fixed 256-byte size of the stack when pushing data onto it.
- Data stored on the stack must be retrieved in the reverse of the order from which it was pushed onto the stack.
- The stack is a **Last-In, First-Out (LIFO)** data structure, meaning when you "pop" a value from the stack, it is the byte most recently pushed onto it. The **PLA** instruction increments the S register by one, then copies the value at the address indicated by the S register (plus the \$100 offset) into the A register.

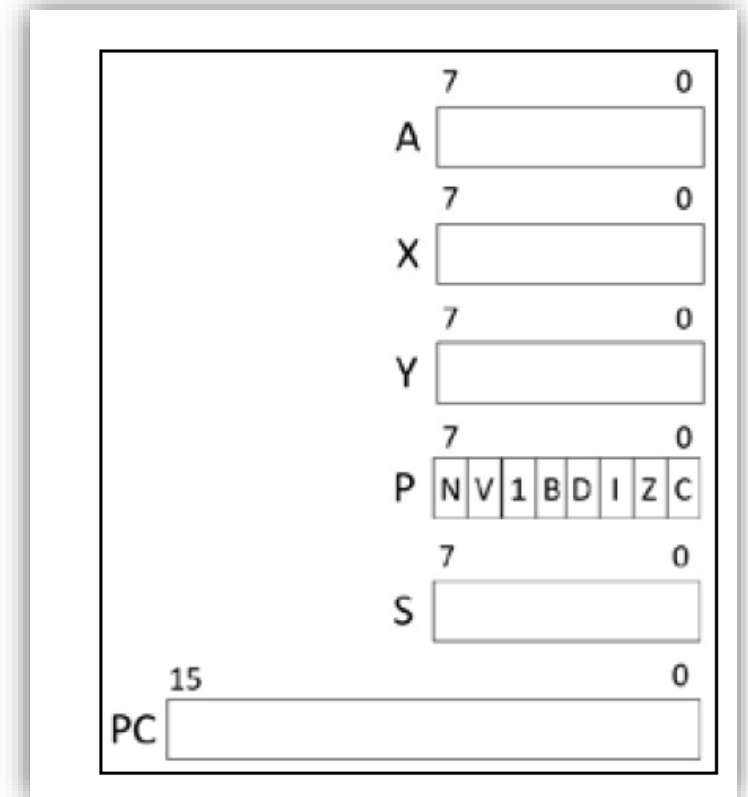


Figure 1.39: overview of 6502 registers

— Computer Architecture

Chapter 1: Introducing Computer Architecture

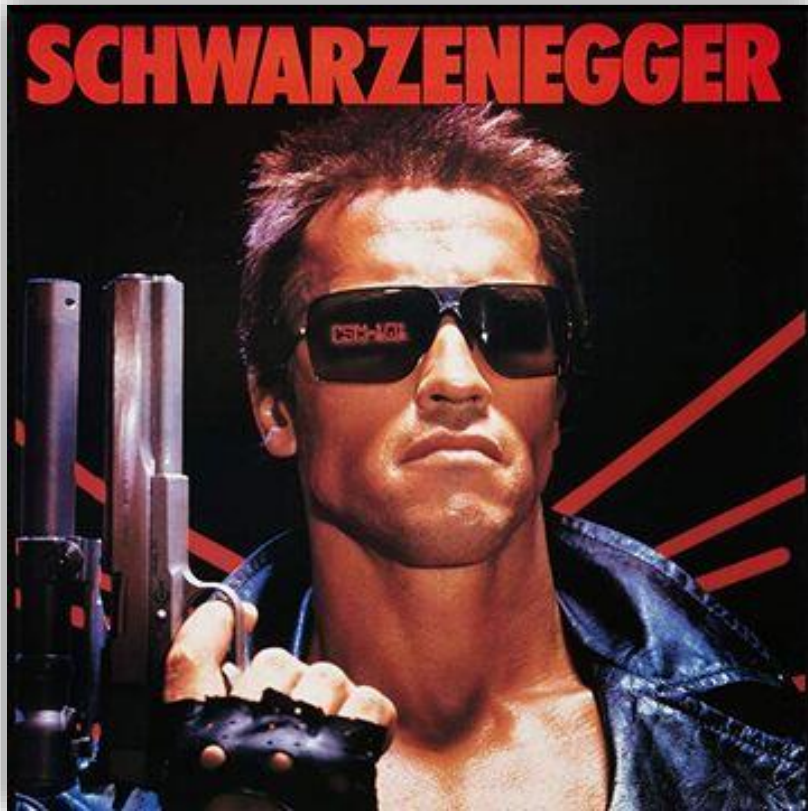


Figure 1.43: The Terminator movie

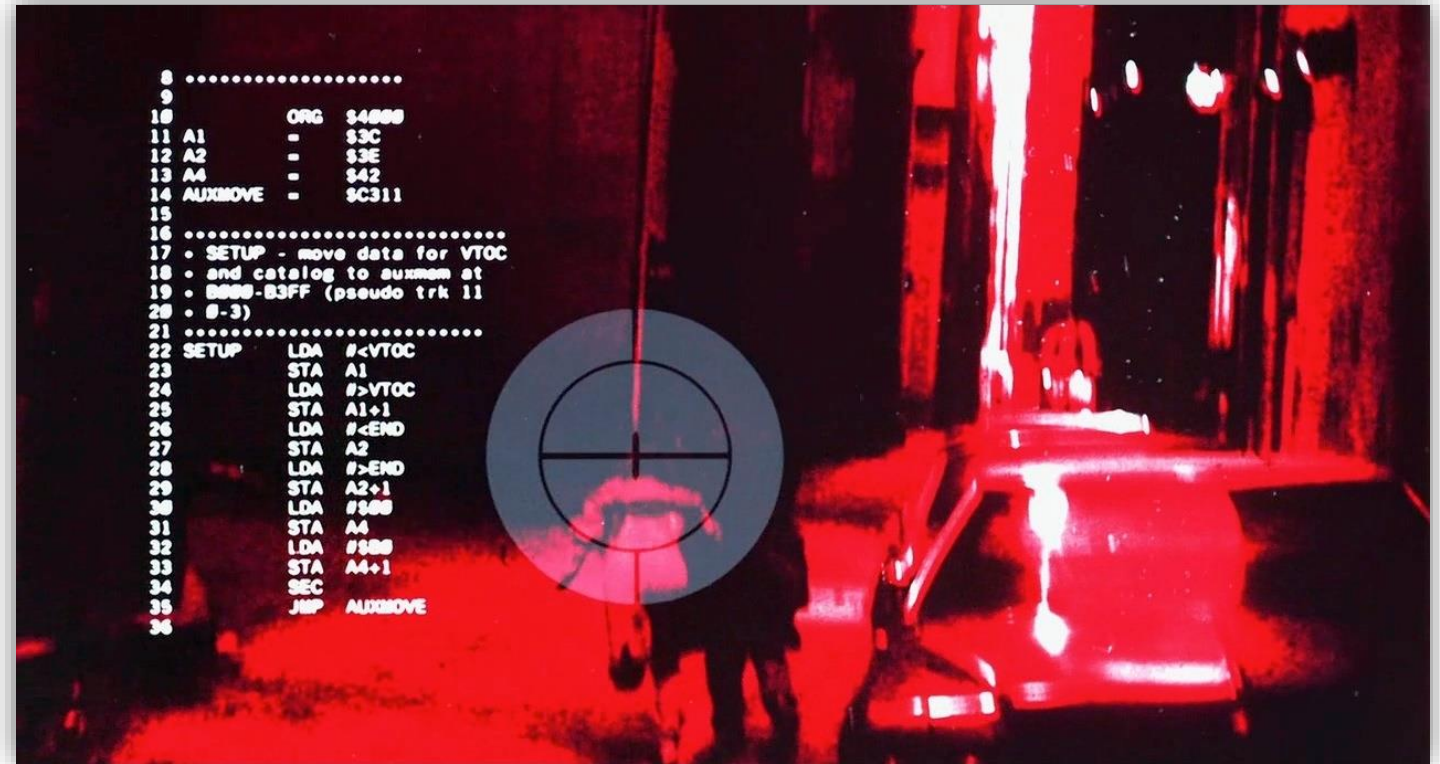


Figure 1.44: 6502 assembly code in The Terminator movie

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 microprocessor

- The **PC register** is the **program counter**.
- This register contains the memory address of the next instruction to be executed. Unlike the other registers, the PC is 16 bits long, allowing access to the entire 6502 address space.
- Each **instruction** consists of a **1-byte operation code**, called **opcode** for short, and may be followed by zero to two operand bytes, depending on the instruction.
- After each instruction executes, the PC updates *to point to the next instruction following the one that just completed*.
- In addition to these automatic updates during sequential instruction execution, the **PC** can be modified by **jump instructions**, **branch instructions**, and **subroutine call** and **return instructions**.

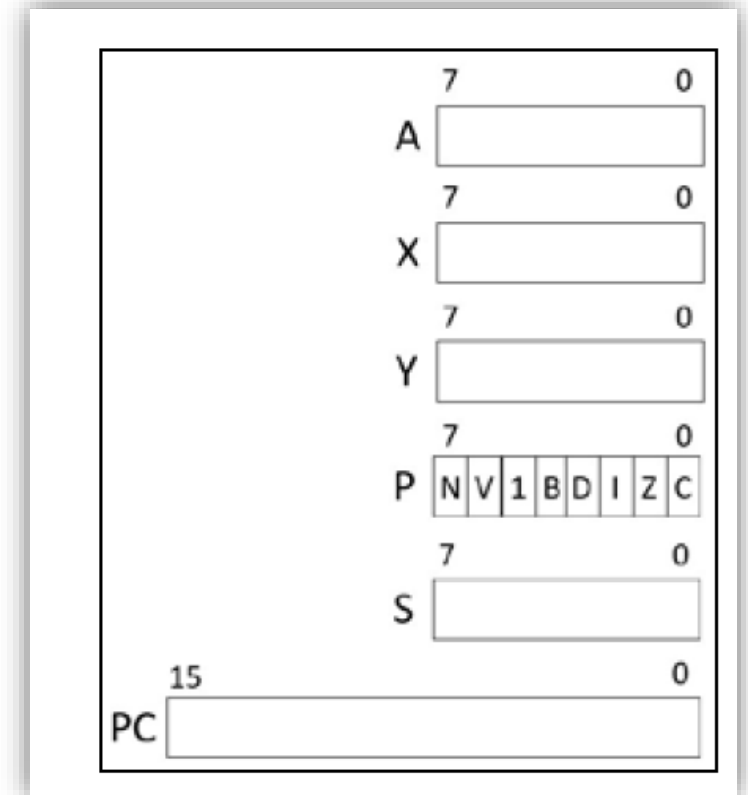


Figure 1.39: overview of 6502 registers

Computer Architecture

Chapter 1: Introducing Computer Architecture

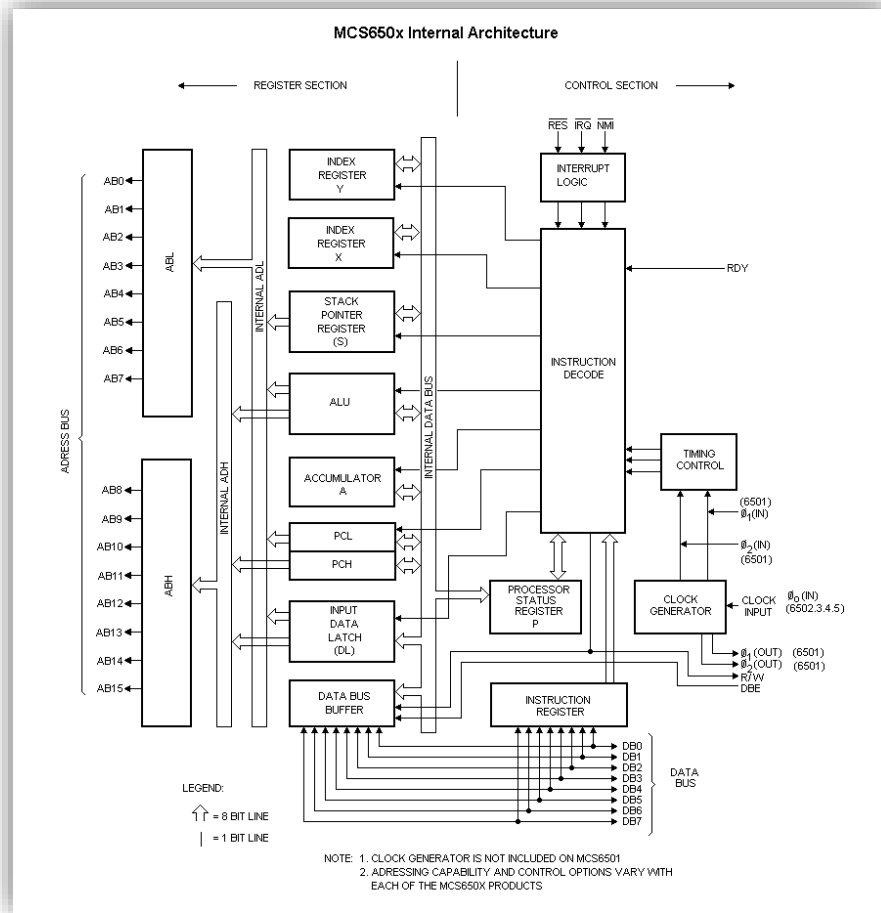


Figure 1.45: 6502 internal architecture

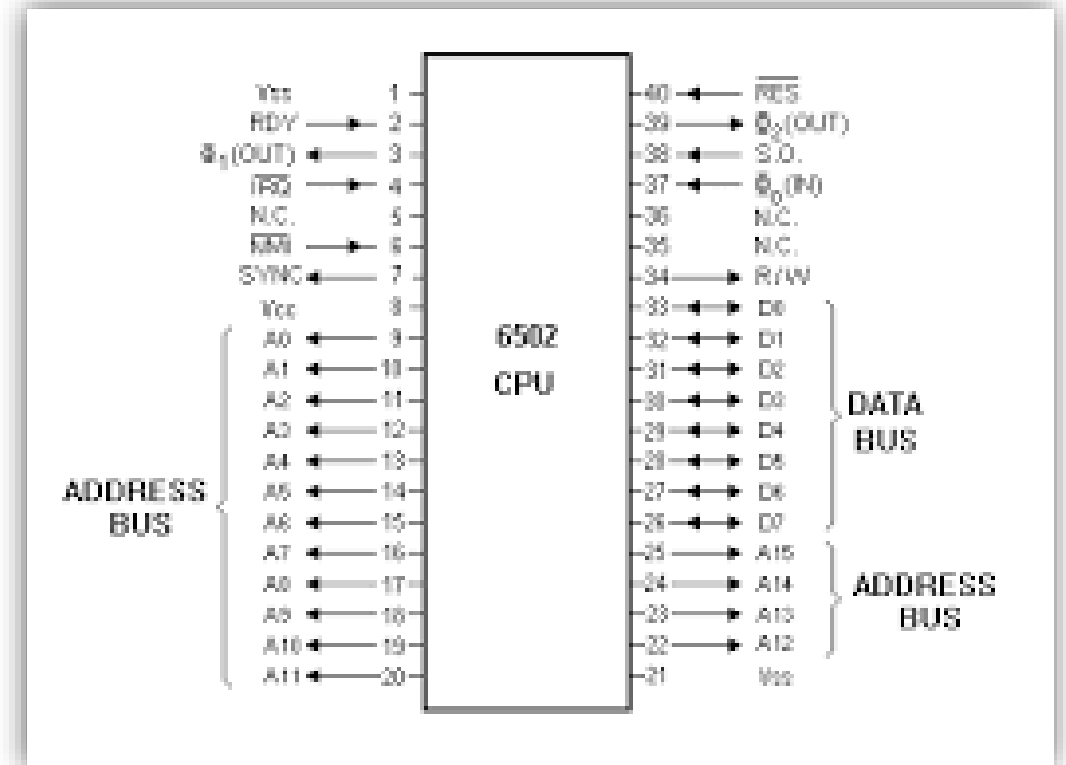


Figure 1.46: 6502 microprocessor pin layout

— **Computer Architecture**

Chapter 1: Introducing Computer Architecture

Contents

1. Introduction
2. The evolution of automated computing devices
 - Charles Babbage's Analytical Engine
 - ENIAC
 - IBM PC
 - The iPhone
3. Moore's law
- 4. Computer architecture**
 - The 6502 microprocessor
 - **The 6502 instruction set**

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 instruction set

- Each of the 6502 instructions has a **three-character mnemonic**. In *assembly* language source files, each line of code contains an instruction **mnemonic** followed by any **operands** associated with the instruction.
- The combination of the mnemonic and the operands defines the **addressing mode**. The 6502 supports **several addressing modes** providing a great deal of flexibility in accessing data in registers and memory.
- For this introduction, we'll only work with the **immediate addressing mode**, in which the operand itself contains a value rather than indicating a register or memory location containing the value. An immediate value is preceded by a **#** character.
- In 6502 assembly, decimal numbers have no special character (48 means 48 decimal) while hexadecimal values are preceded by a **\$** character (\$30 means 30 hexadecimal, equivalent to 00110000b and to 48 decimal).
- An immediate *decimal* value looks like **#48** and an immediate *hexadecimal* value looks like **#\$30**.
- Five 6502 instructions are used in the following examples:
 - **LDA** **loads** register A with a value.
 - **ADC** performs **addition** using the **Carry** (C flag) as an additional input and output.
 - **SBC** performs **subtraction** using the **Carry** flag as an additional input and output.
 - **SEC** **sets** the **Carry** flag directly.
 - **CLC** **clears** the **Carry** flag directly.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 instruction set

- Since the **Carry** flag is an input to the addition and subtraction instructions, it is important to ensure it has the correct value prior to executing the **ADC** or **SBC** instructions. *Before initiating an addition operation, the C flag must be clear to indicate there is no carry from a prior addition.*
- When performing multi-byte **additions** (say, with 16-bit, 32-bit, or 64-bit numbers), the carry, if any, will **propagate** from the sum of one byte pair to the next as you add the more significant bytes to each other.
- If the C flag is **set** when the ADC instruction executes, the effect is to add one to the result.
- After the ADC completes, the C flag serves as the *ninth bit of the result*: a C flag result of 0 means there was no carry, and a 1 indicates there was a carry from the 8-bit register.
- In the 6502, the C flag represents the **opposite of Borrow**.
- *If C is 1, then Borrow is 0, and if C is 0, Borrow is 1.*
- Performing a simple **subtraction** with **no** incoming **Borrow** (B=0) requires **setting** the C flag (C=1) before executing the **SBC** command.
- The examples in the following employ the 6502 as a calculator using inputs defined directly in the code and with the result stored in the **A register**.
- The Results columns show the final value of the A register and the N, V, Z, and C flags.

— Computer Architecture

Chapter 1: Introducing Computer Architecture

Instruction Sequence	Description	Results				
		A	N	V	Z	C
CLC LDA #1 ADC #1	8-bit addition with no Carry input: Clear the Carry flag, then load an immediate value of 1 into the A register and add 1 to it.	\$02	0	0	0	0
SEC LDA #1 ADC #1	8-bit addition with a Carry input: Set the Carry flag, then load an immediate value of 1 into the A register and add 1 to it.	\$03	0	0	0	0
SEC LDA #1 SBC #1	8-bit subtraction with no Borrow input: Set the Carry flag, then load an immediate value of 1 into the A register and subtract 1 from it. C = 1 indicates no borrow occurred.	\$00	0	0	1	1

CLC LDA #1 SBC #1	8-bit subtraction with a Borrow input: Clear the Carry flag, then load an immediate value of 1 into the A register and subtract 1 from it. C = 0 indicates a borrow occurred.	\$FF	1	0	0	0
CLC LDA #\$FF ADC #1	Unsigned overflow: Add 1 to \$FF. C = 1 indicates a carry occurred.	\$00	0	0	1	1
SEC LDA #0 SBC #1	Unsigned underflow: Subtract 1 from 0. C = 0 indicates a borrow occurred.	\$FF	1	0	0	0
CLC LDA #\$7F ADC #1	Signed overflow: Add 1 to \$7F. V = 1 indicates signed overflow occurred.	\$80	1	1	0	0
SEC LDA #\$80 SBC #1	Signed underflow: Subtract 1 from \$80. V = 1 indicates signed underflow occurred.	\$7F	0	1	0	1

Figure 1.47: 6502 arithmetic instruction sequences

— Computer Architecture

Chapter 1: Introducing Computer Architecture

4. Computer architecture: The 6502 architecture

- JavaScript 6502 assembler and simulator: <https://skilldrick.github.io/easy6502/>

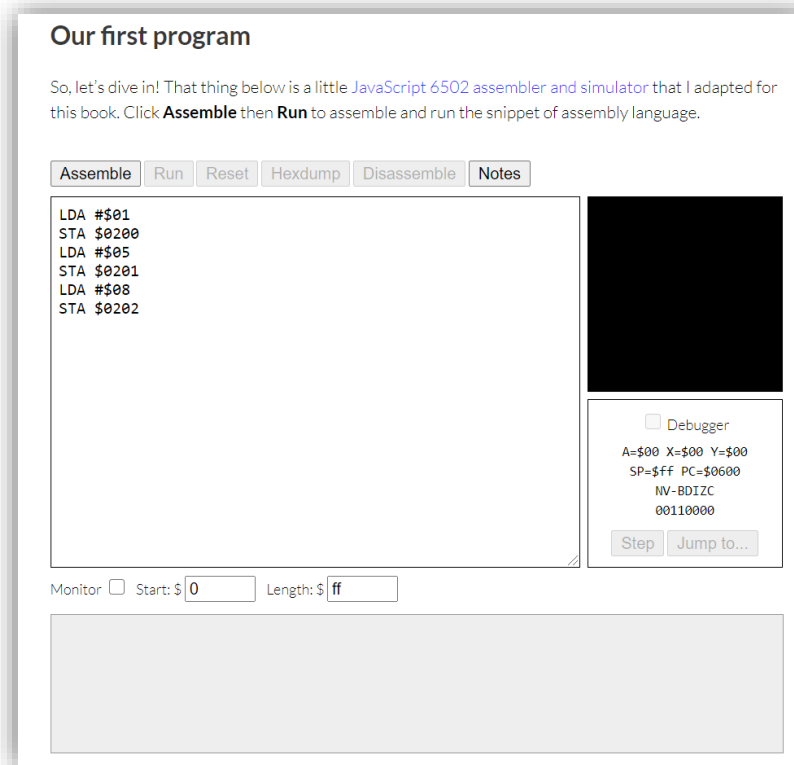
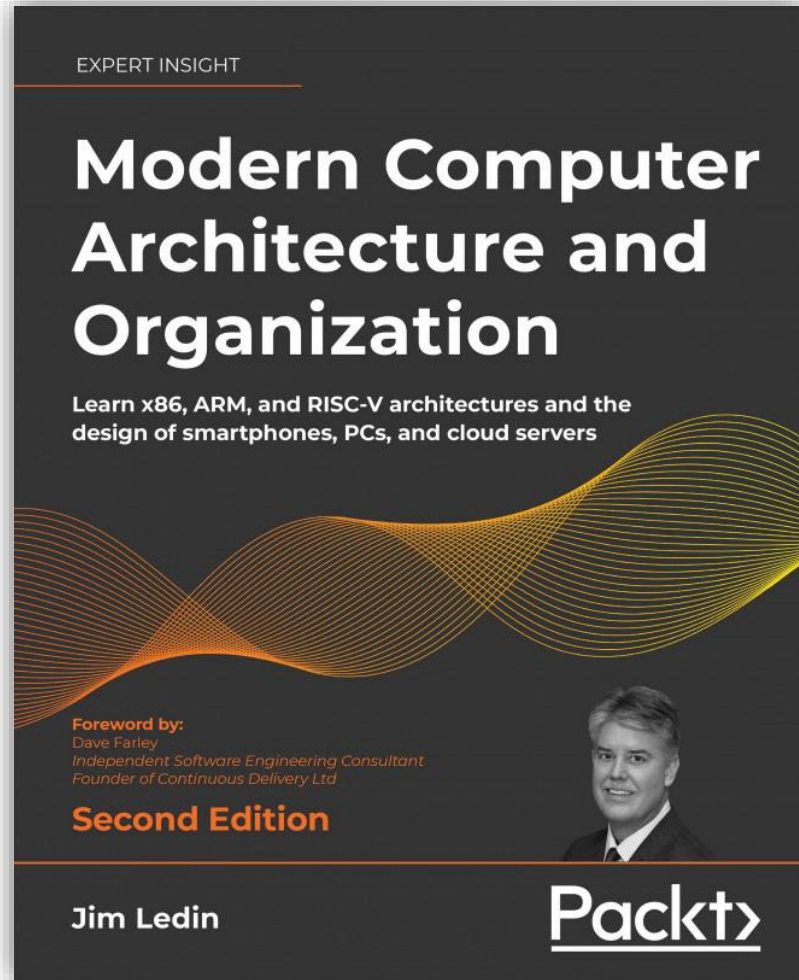


Figure 1.48: JavaScript 6502 assembler and simulator

Computer Architecture

End of Chapter 1 – Introducing Computer Architecture



VIVES University of Applied Sciences
Bachelor in electronics-ICT

